Mount Kenya University

Unlocking Infinite Possibilities

Student Name: Kenan Kiplimo

Admission Number: BSCCS/2025/67071

Course/Class: Bachelor of Science in Computer science

Unit Code: BIT4138

Unit Name: Advanced Cryptography

Lecturer Name: Mr. Michael Nyoro

Semester/Academic Year: Year 3 Semester 2, May 2026

Selected Technologies:

- ParrotOS Security
- VSCodium
- Python

GitHub: https://github.com/Kynan51/advanced-cryptography

Week 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment

Fig 1: Installation of VSCodium

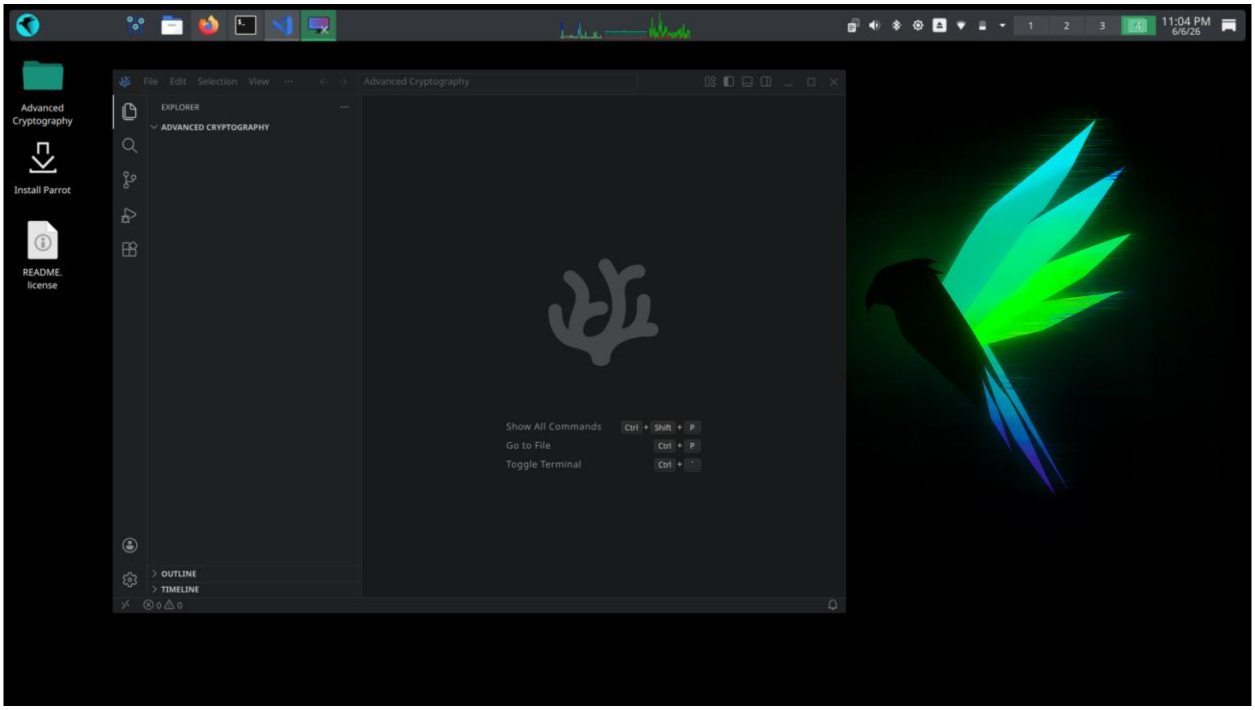


Fig 2: Installation of Cryptography Libraries

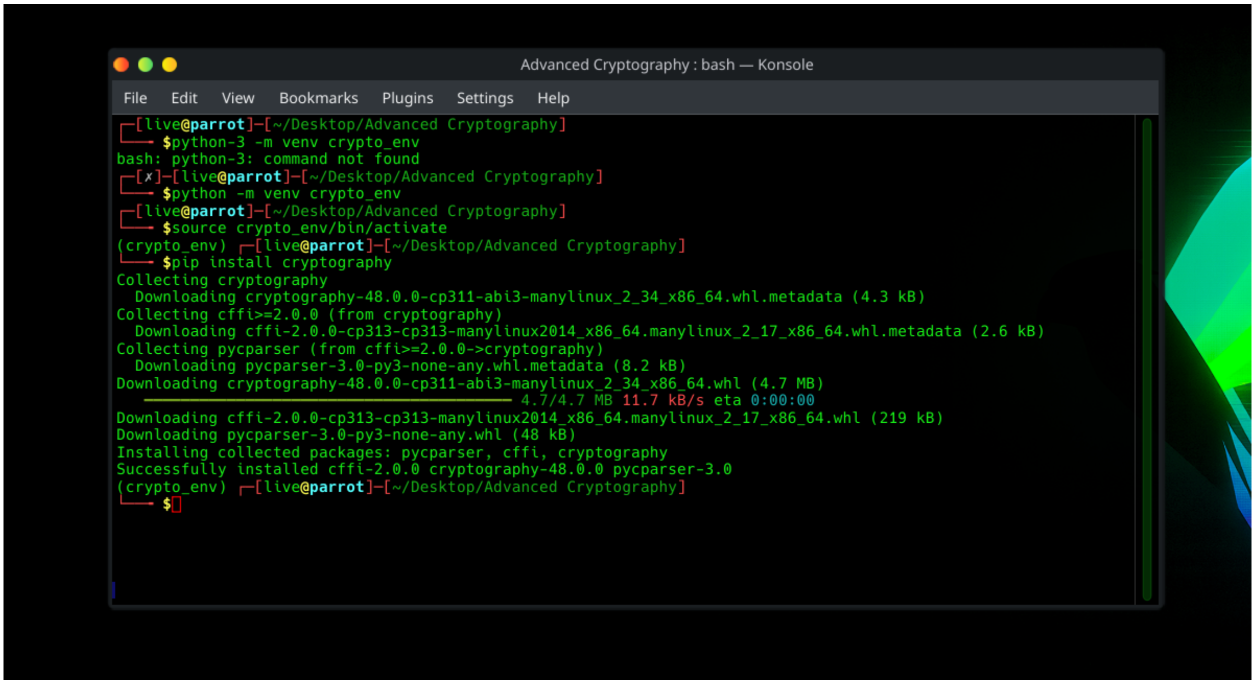


Fig 3: OpenSSL Installation Test

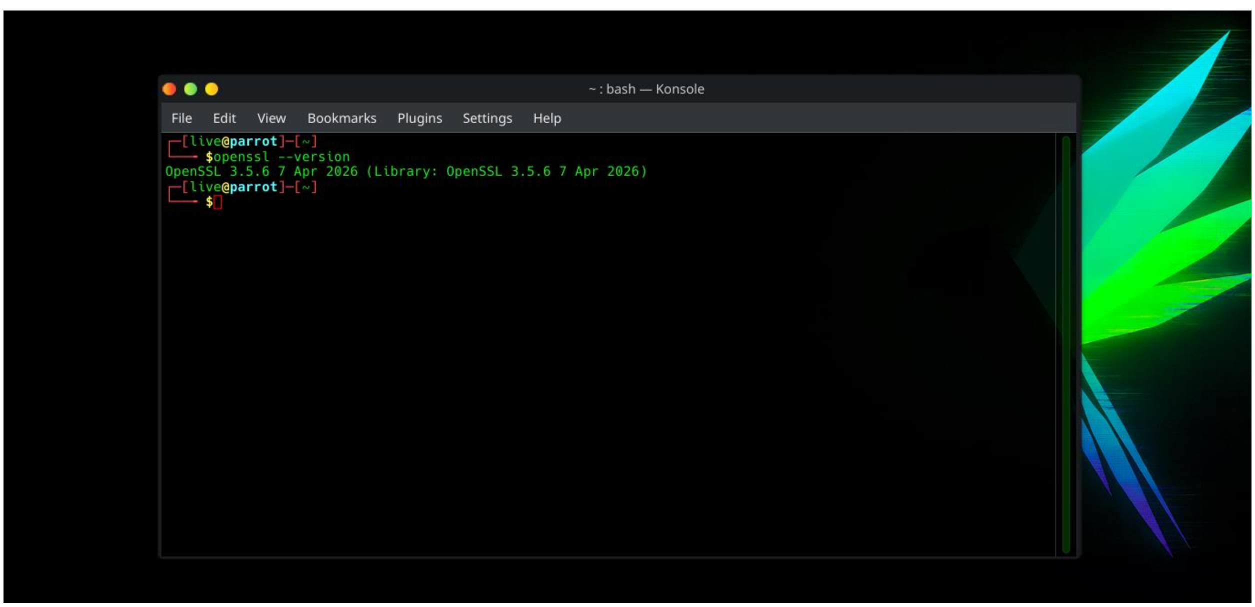


Fig 4: Local Encryption Script Execution

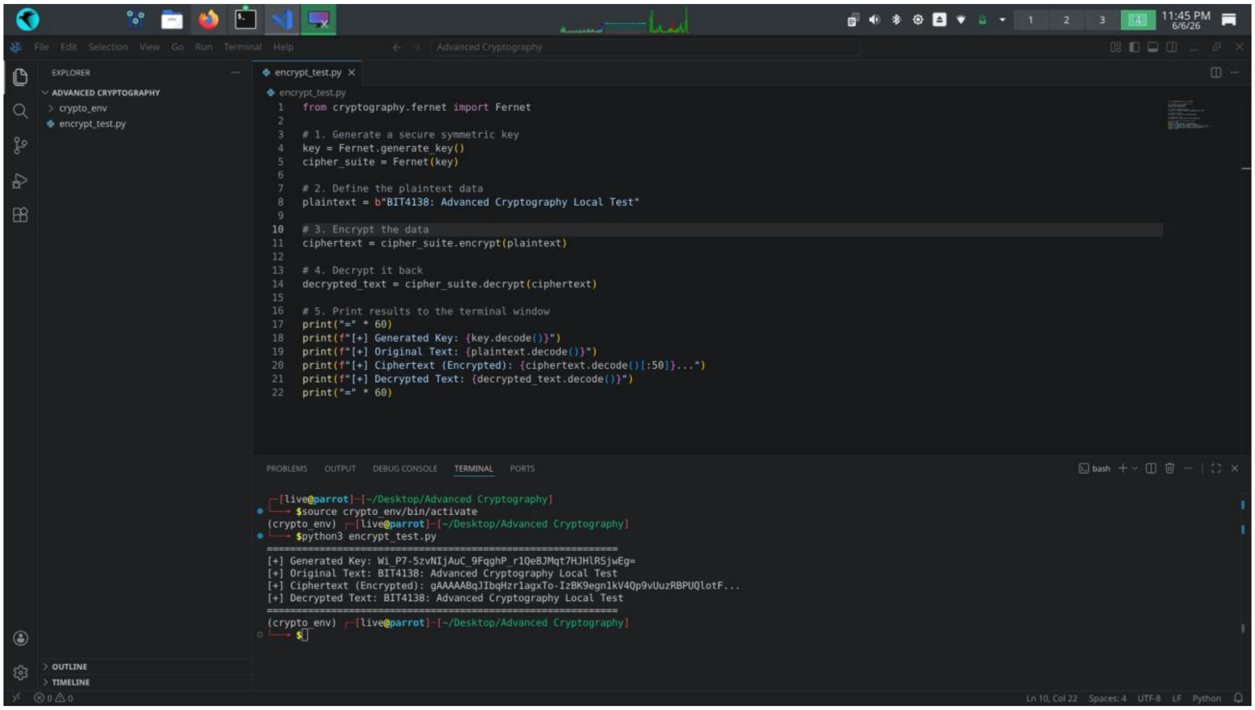
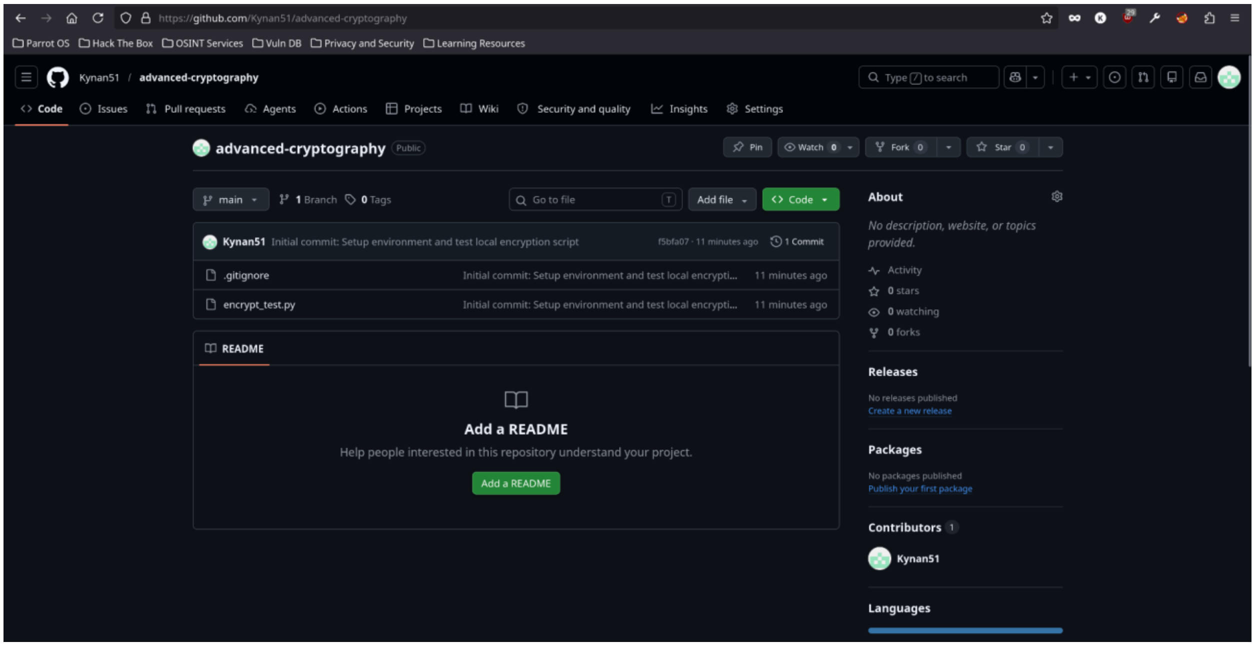


Fig 5: GitHub Repository Initialization



Student Reflection

Configured a portable development environment on a live persistent Parrot OS workstation. Verified the pre-installed OpenSSL architecture and established secure version control via Git. Fig 3 confirms successful installation and configuration of OpenSSL used for generating secure cryptographic keys and certificates.

Week 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

Fig 1: Caesar Cipher Implementation

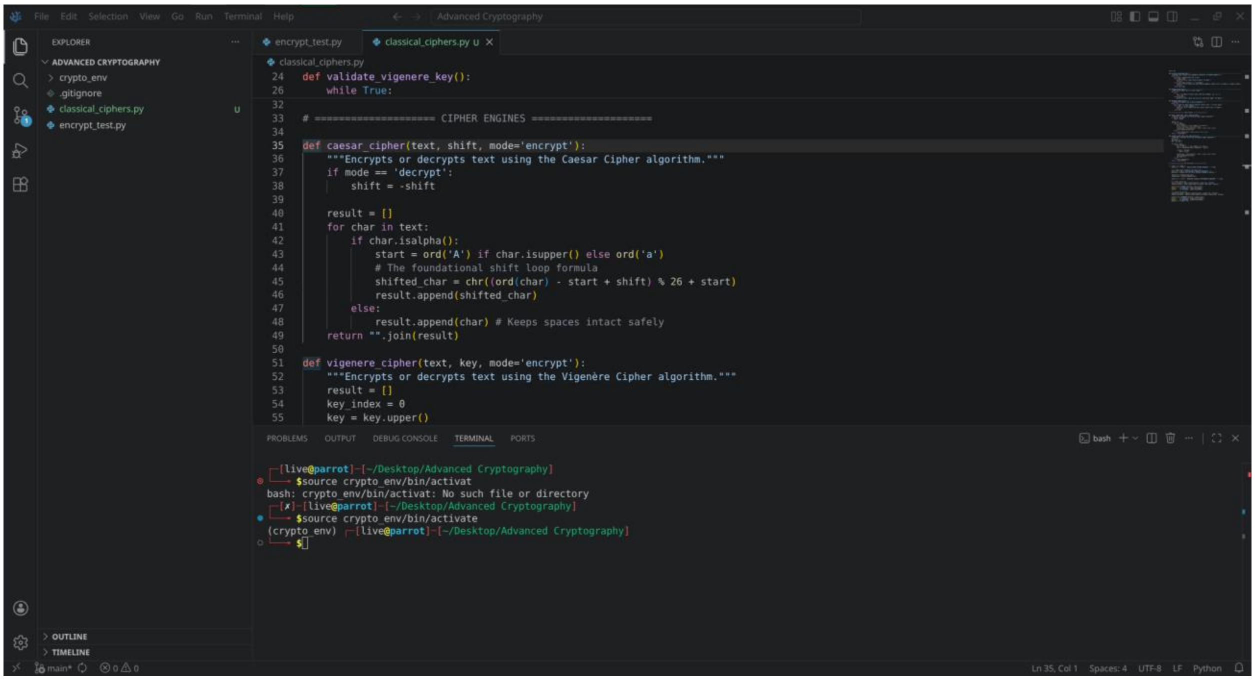


Fig 2: Vigenère Cipher Encryption Process

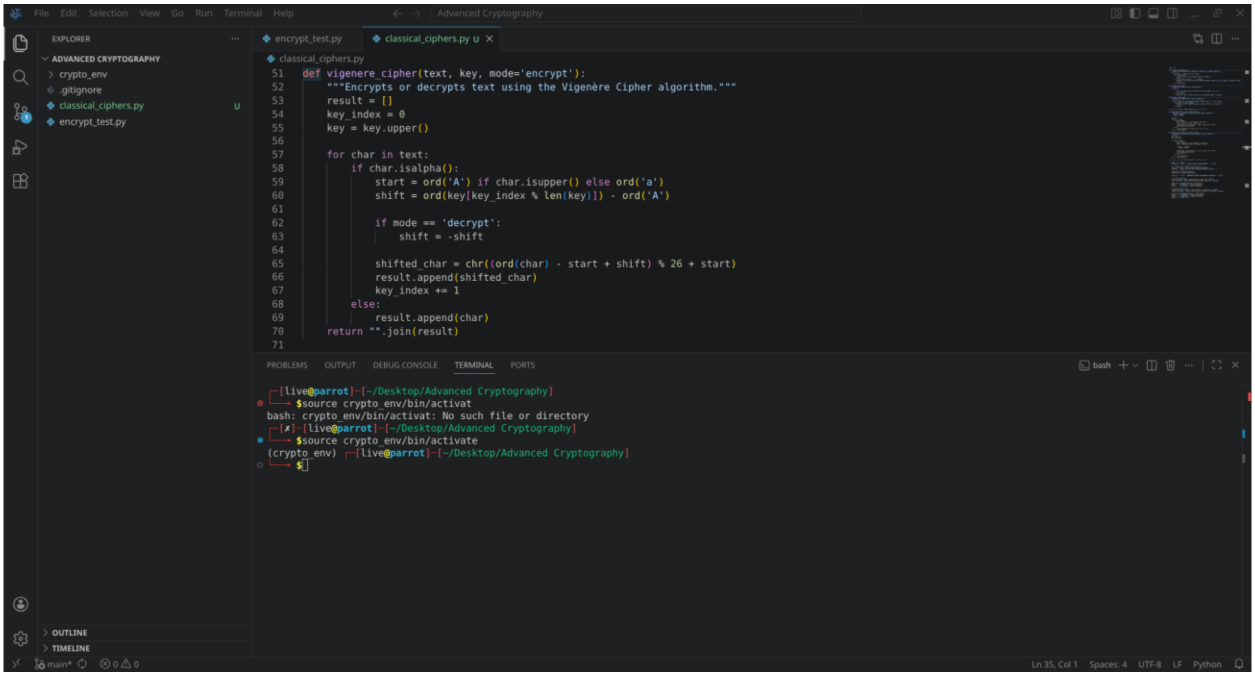


Fig 3: Encryption and Decryption Output

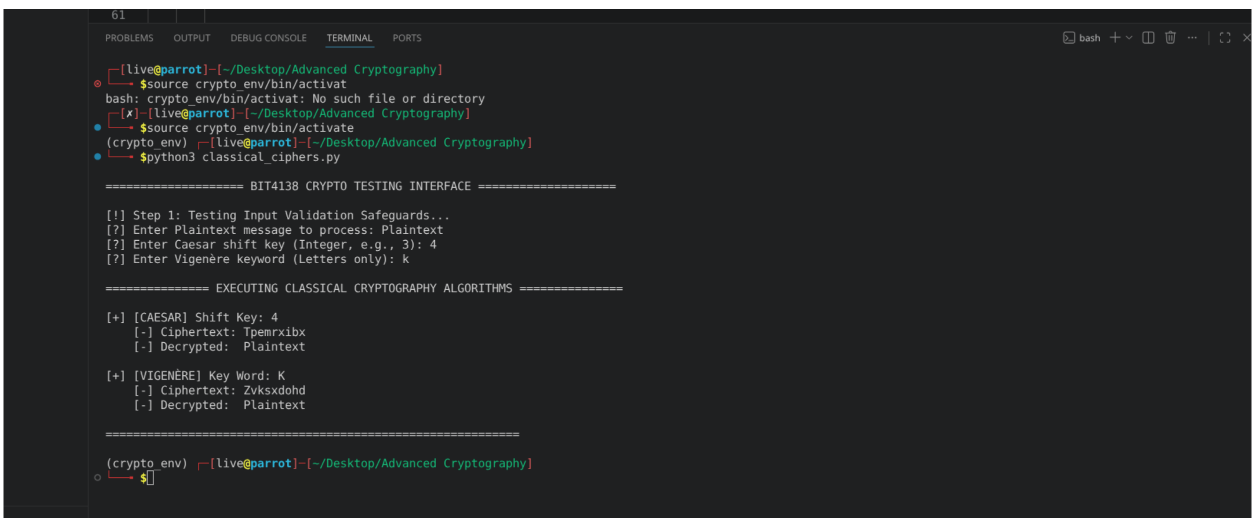


Fig 4: User Input Validation Interface

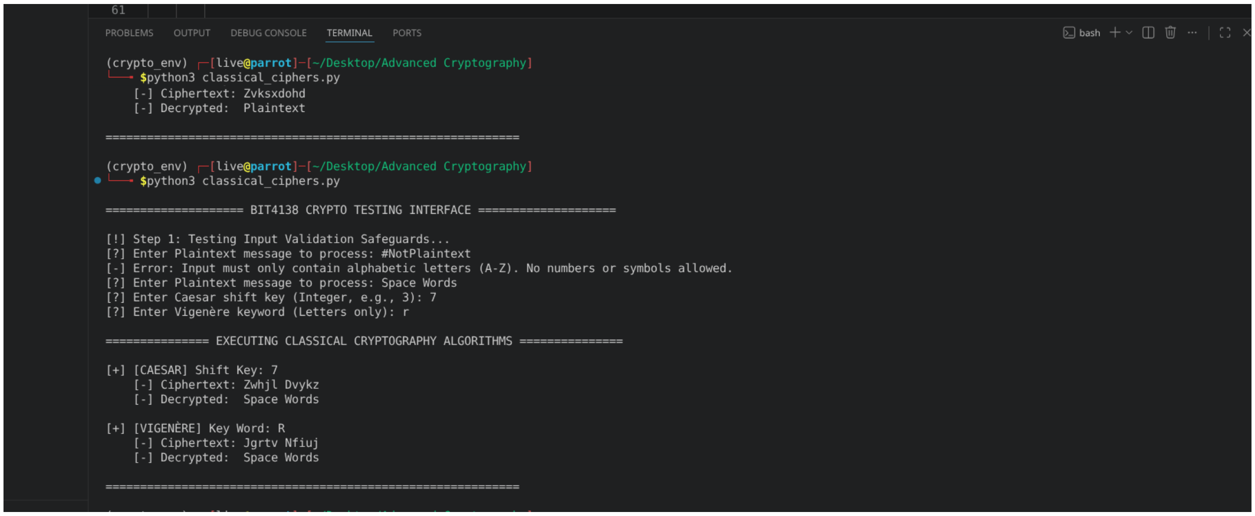
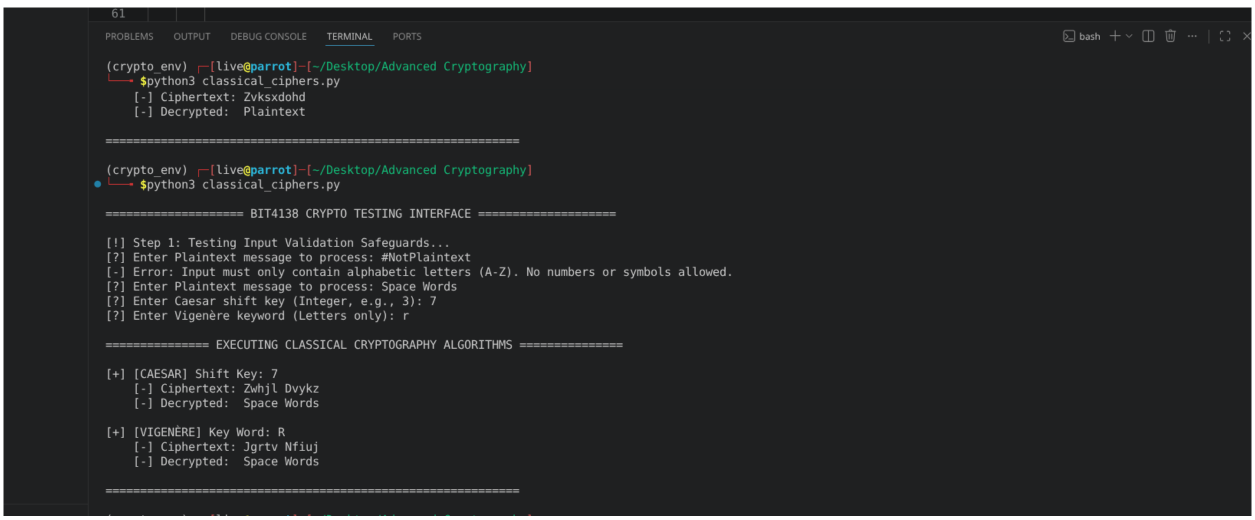


Fig 5: Cipher Testing Results



Student Reflection

Implemented functional Caesar and Vigenère polyalphabetic cipher architectures inside Parrot OS to analyze traditional data transformation rules. Developed input sanitization checks to handle structured data constraints defensively. The Vigenère implementation successfully demonstrated how using a variable alphabet rotation counter flattens pattern distributions to mitigate basic statistical frequency cryptanalysis vulnerabilities

Week 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

Fig 1: LFSR Generator Implementation

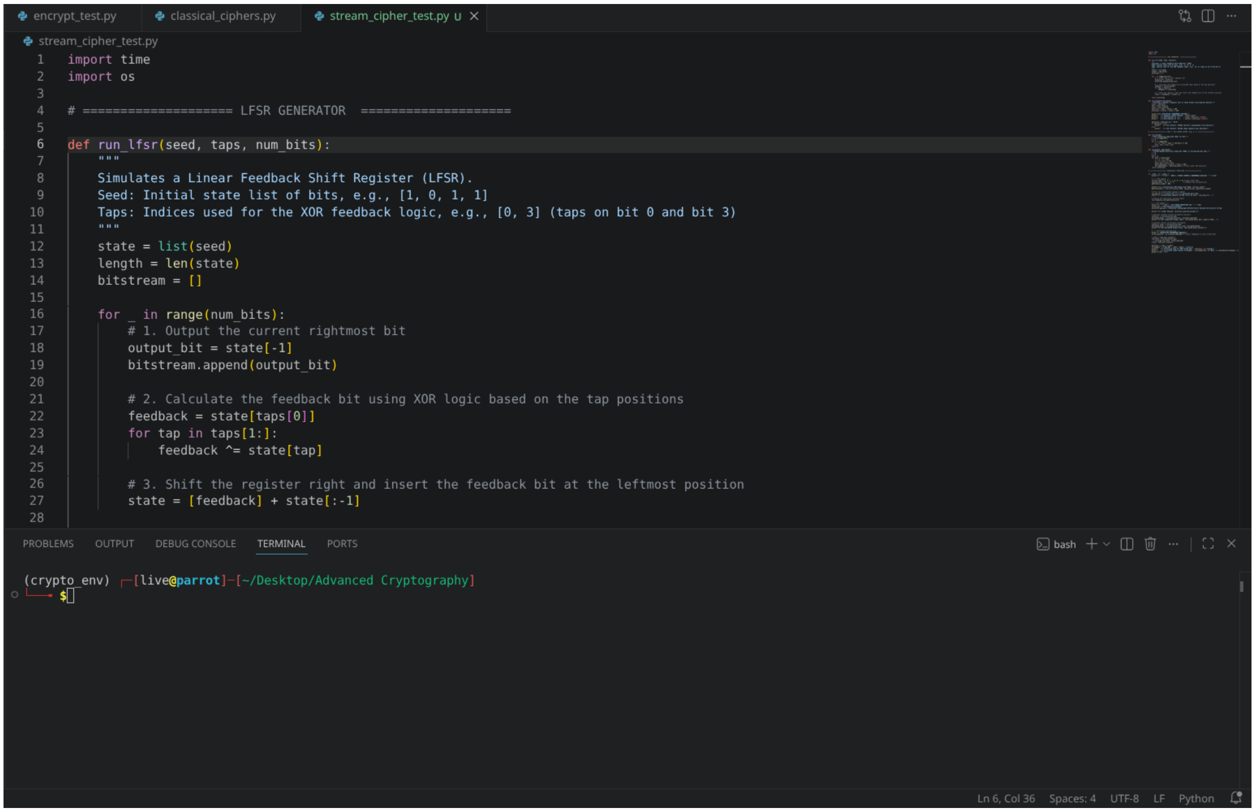


Fig 2: Pseudorandom Sequence Output

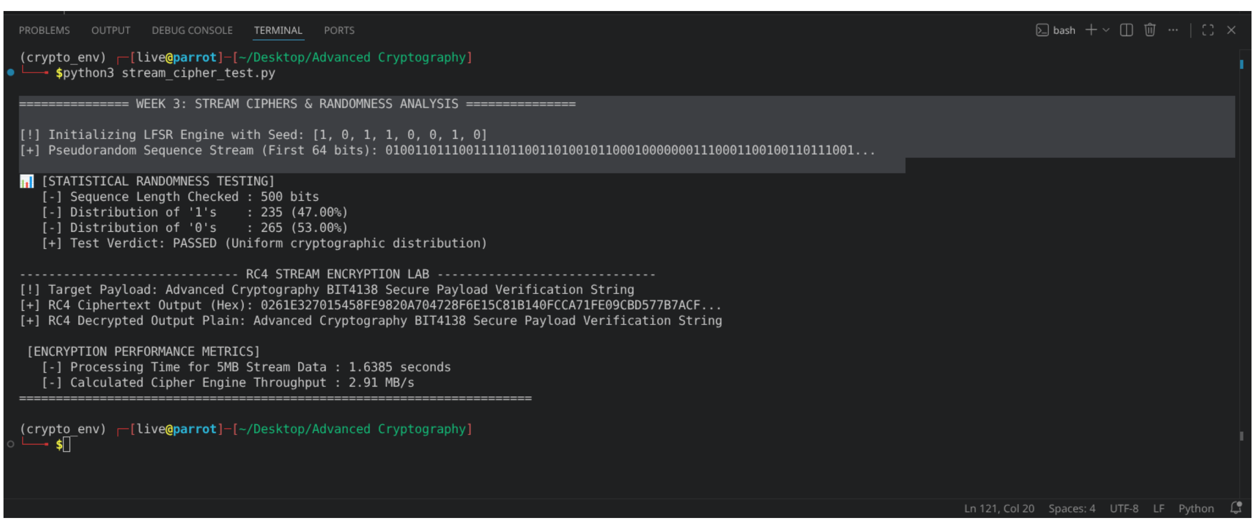


Fig 3: Statistical Randomness Testing

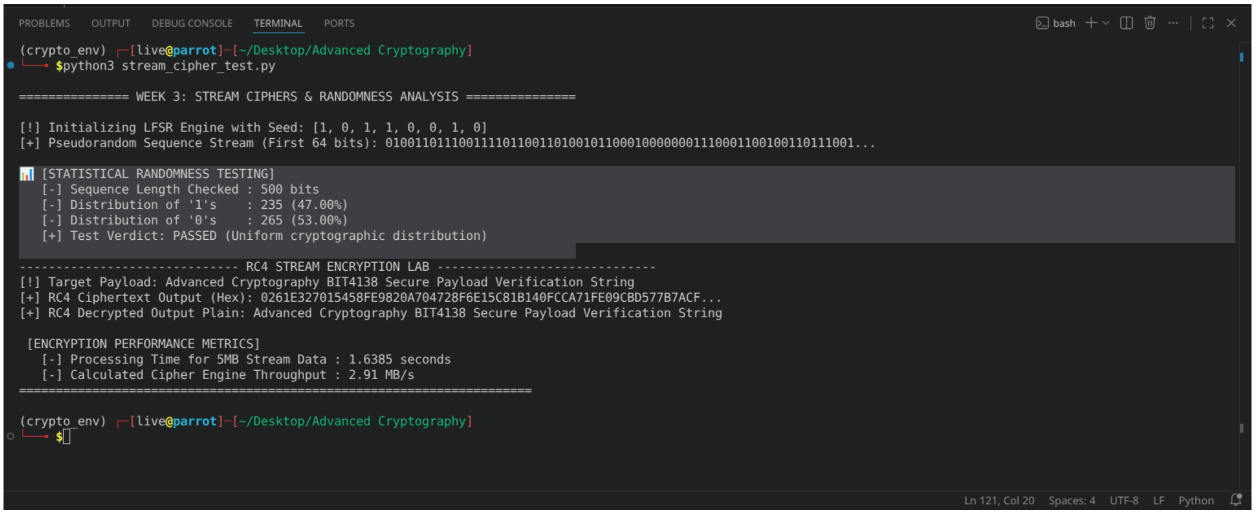


Fig 4: RC4 Stream Cipher Simulation

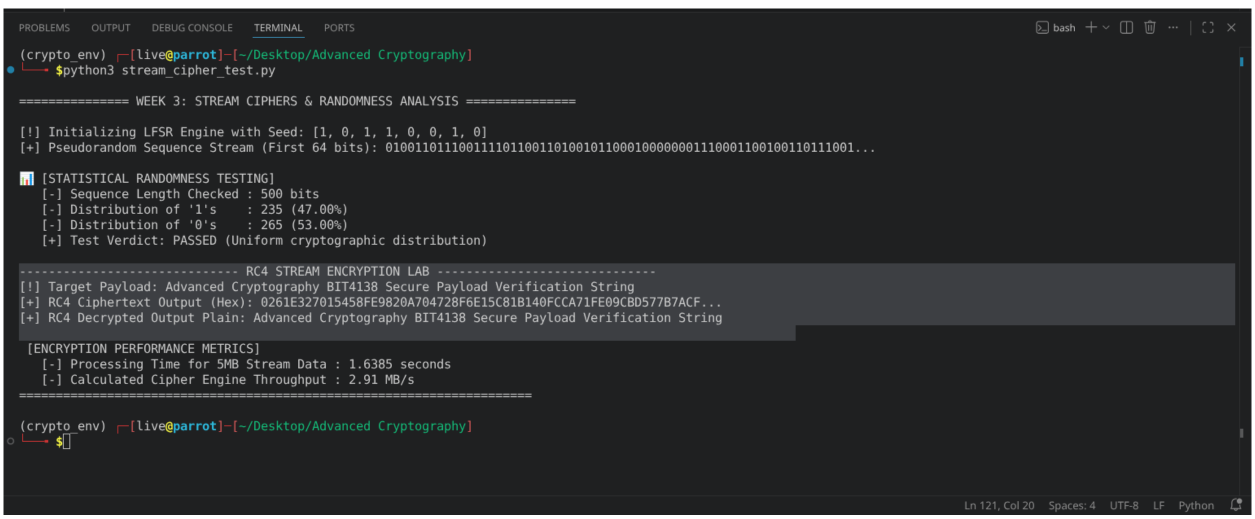
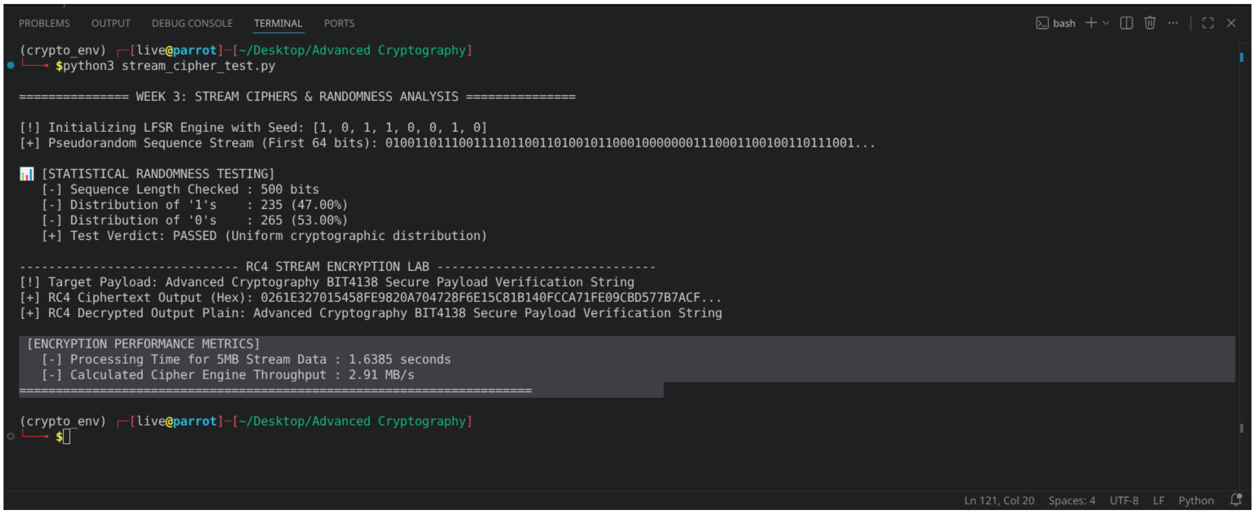


Fig 5: Encryption Performance Results



Student Reflection

Simulated an LFSR and RC4 engine to examine continuous byte-level encryption. Cryptographic randomness creates unpredictable key streams to defend against plaintext structural recognition. Executed monobit statistical testing to verify uniform distribution balance, confirming that high-speed throughput and low latency are successfully achieved without introducing mathematical sequence patterns.

Week 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

Fig 1: AES Encryption Script

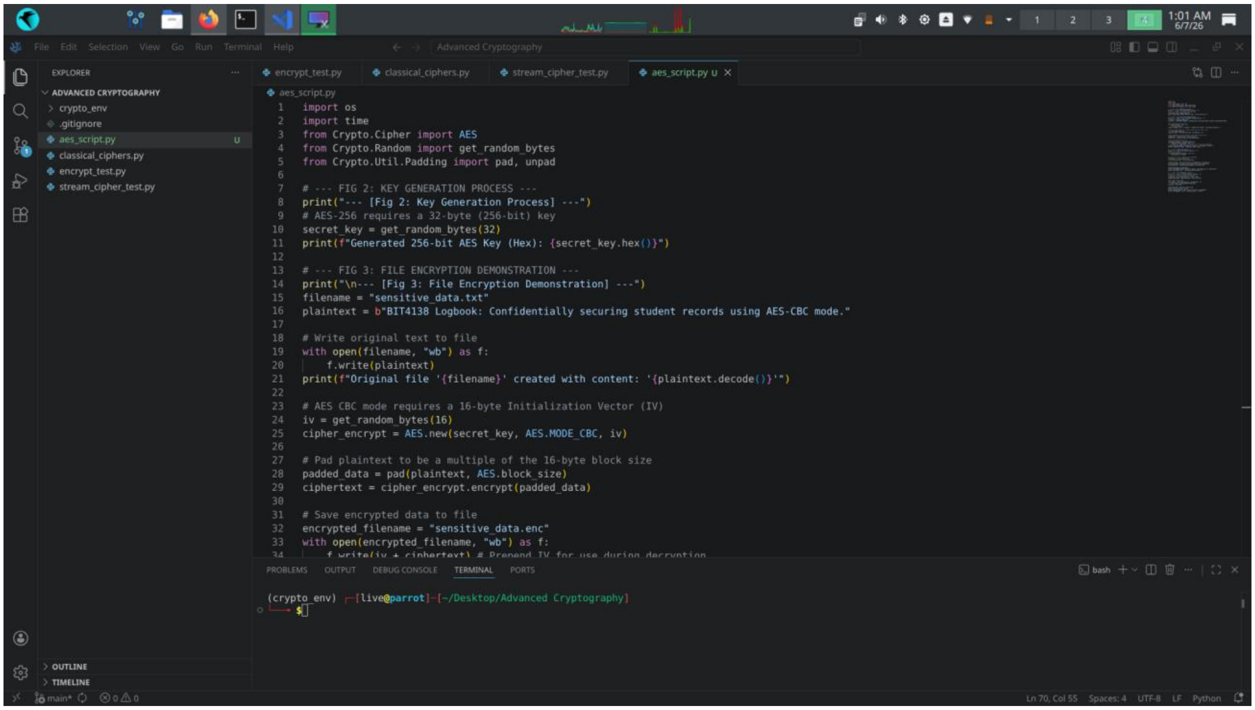


Fig 2: Key Generation Process

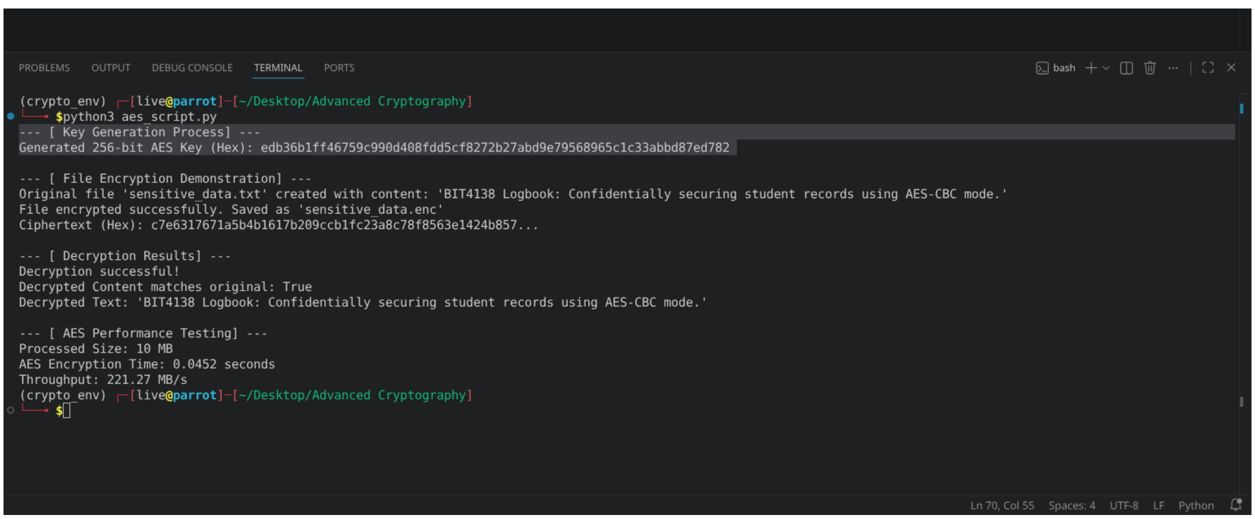


Fig 3: File Encryption Demonstration

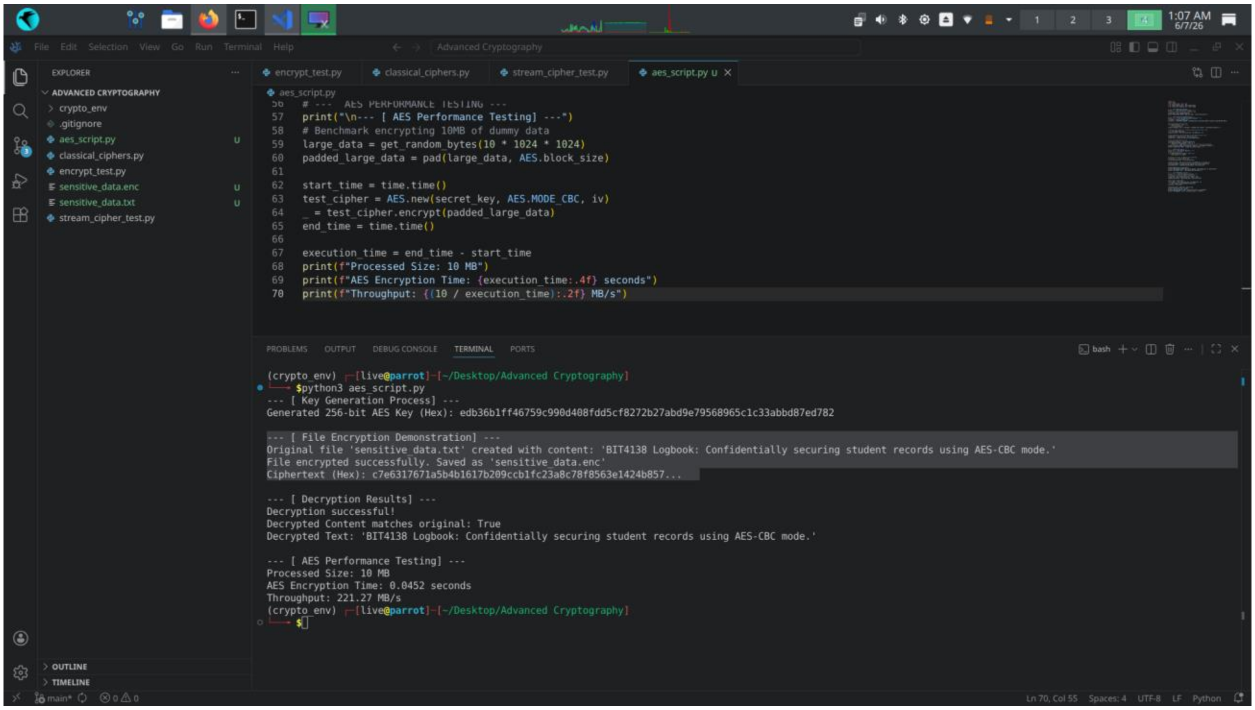


Fig 4: Decryption Results

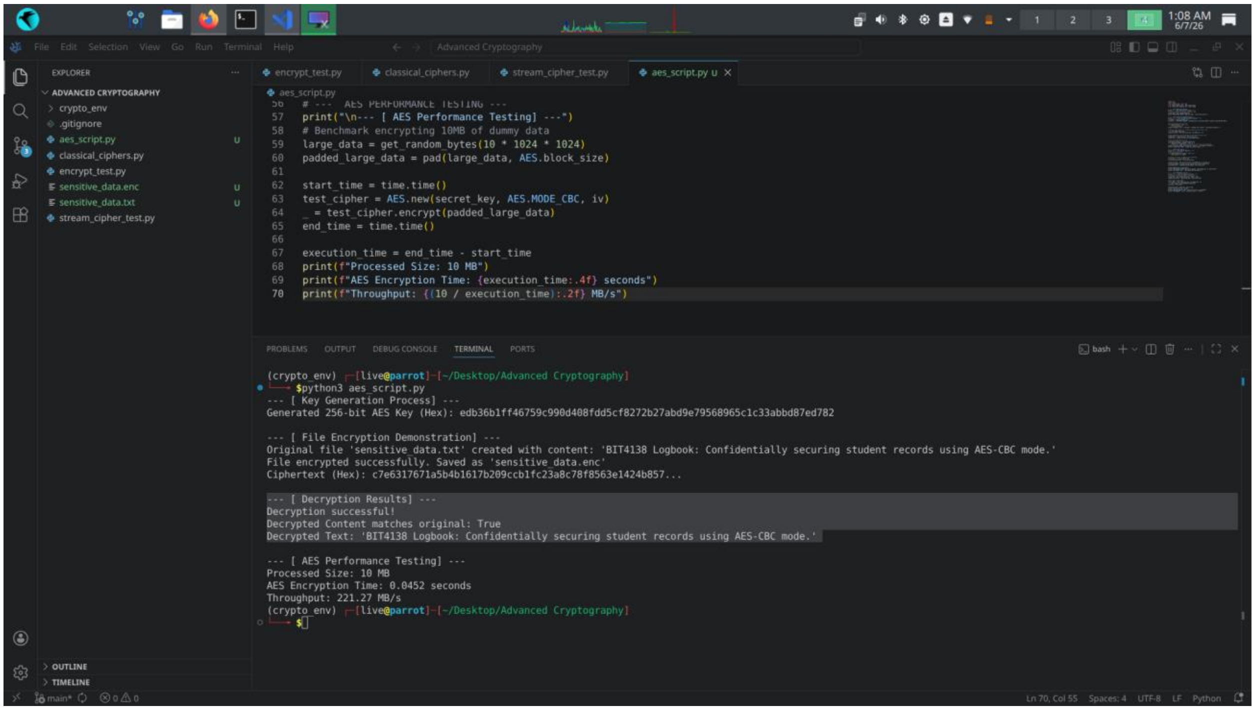
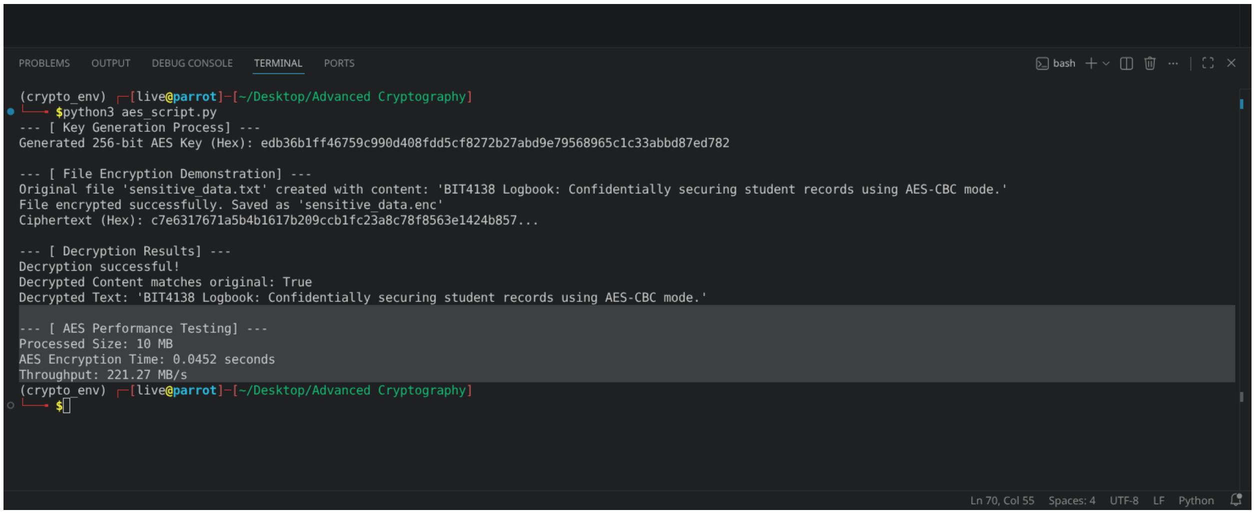


Fig 5: AES Performance Testing



Student Reflection

Implemented AES-256 in CBC mode, mastering its block-by-block diffusion workflow. Correctly padding data and securely handling the initialization vector (IV) was challenging. Secure key management proved vital, as compromised keys invalidate the entire encryption/decryption integrity. Performance testing highlighted AES's exceptional, hardware-optimized computational efficiency with large files.

Week 5: Public Key Cryptography (RSA)

Implementation of RSA Encryption and Key Management

Fig 1: RSA Key Pair Generation

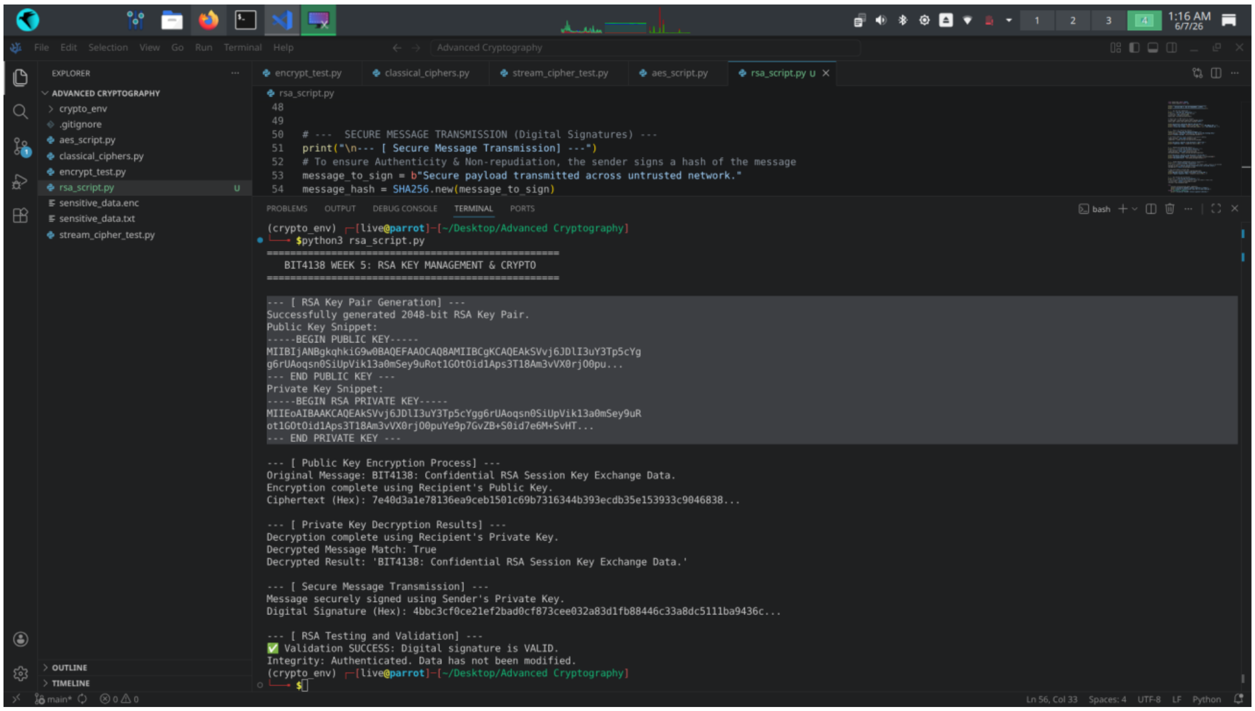


Fig 2: Public Key Encryption Process

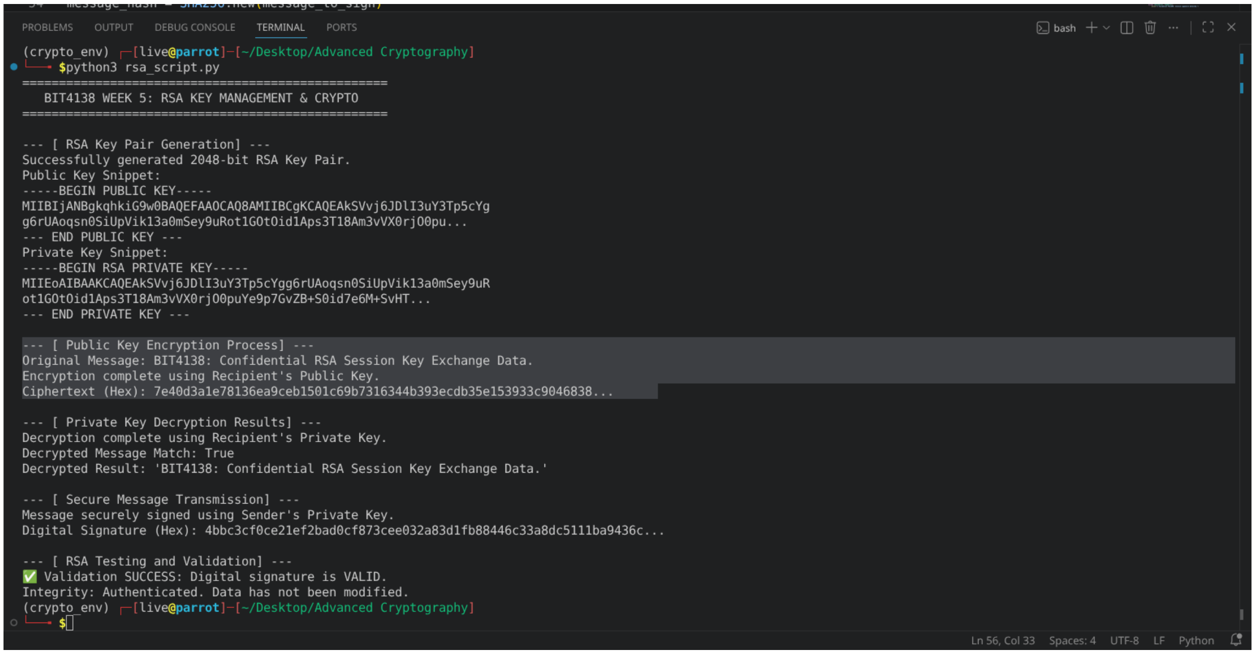


Fig 3: Private Key Decryption Results

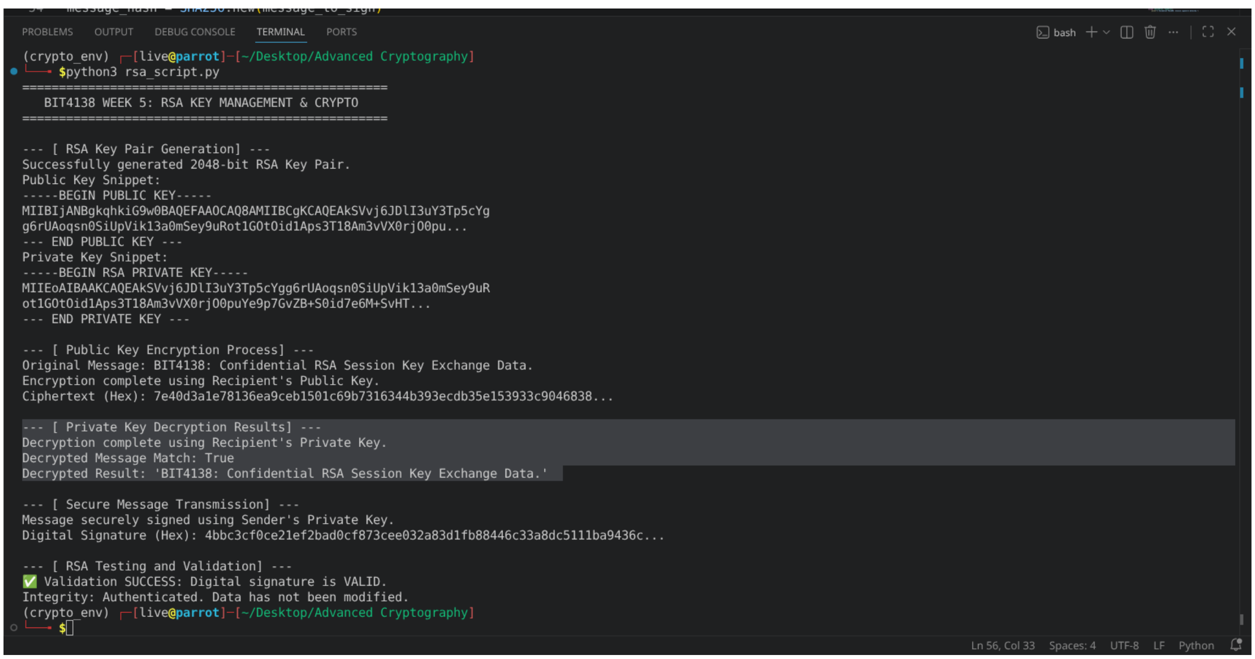


Fig 4: Secure Message Transmission

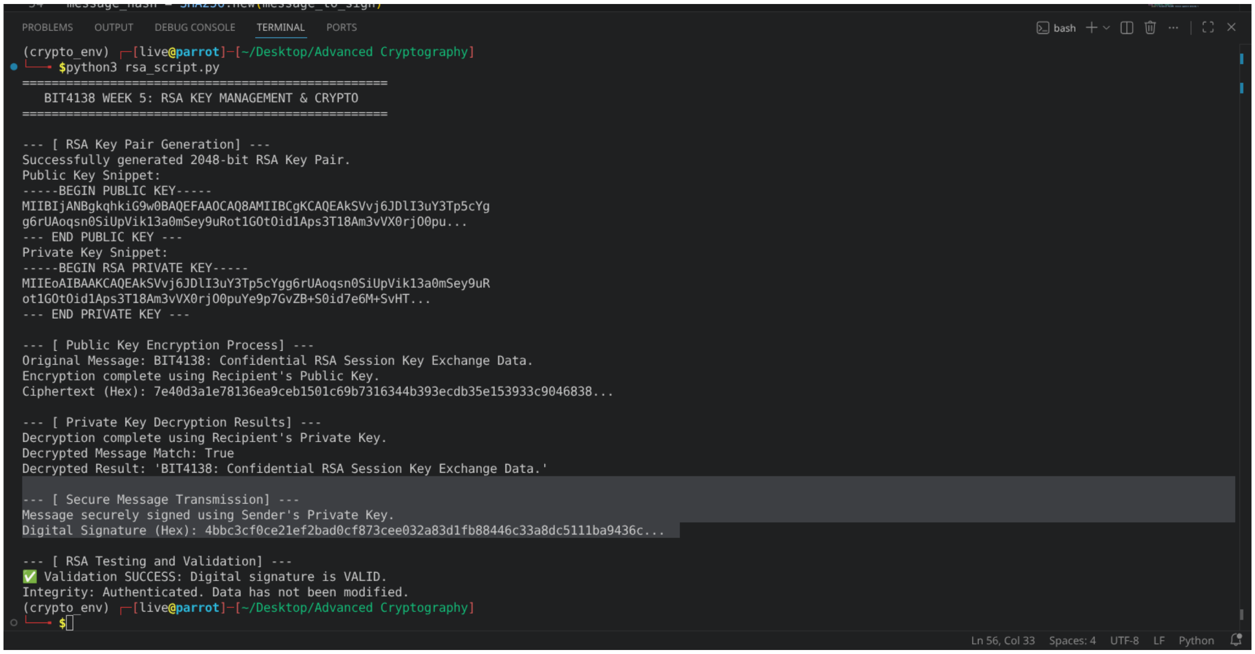
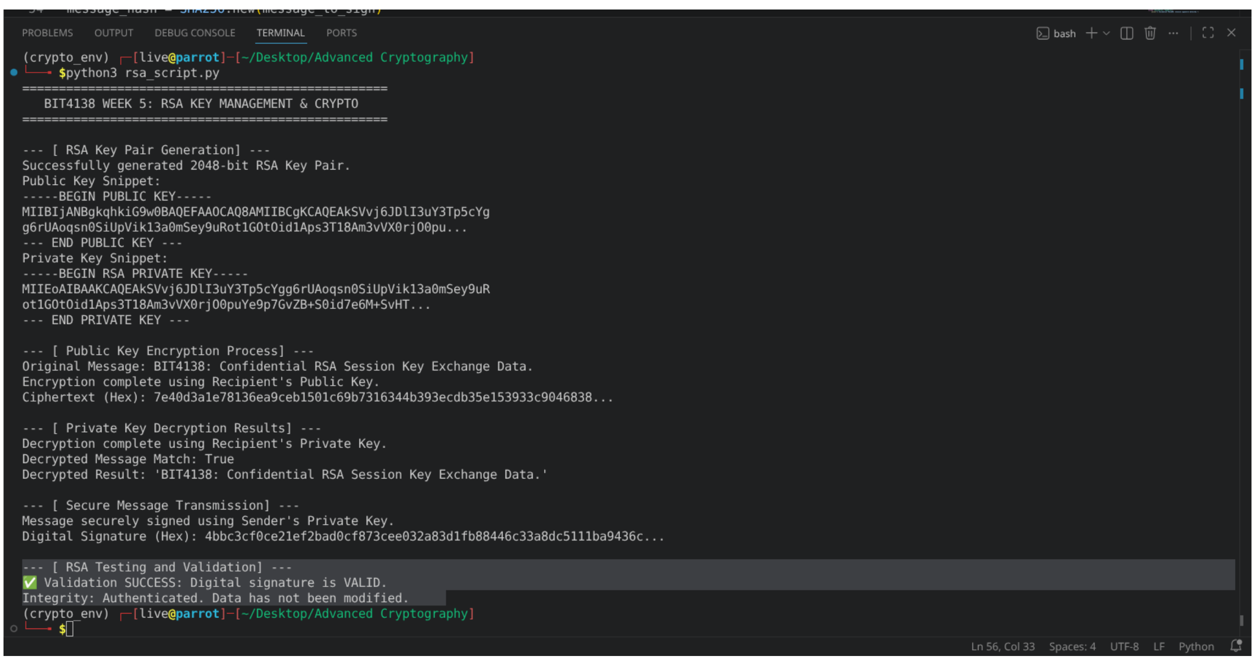


Fig 5: RSA Testing and Validation



Student Reflection

Configured an asymmetric 2048-bit RSA key pair to evaluate public-key infrastructure. I leveraged the public key for data encryption and digital signature verification, while reserving the private key for decryption. Managing mathematical overhead and ensuring signature validation padding configurations were challenging, yet the process perfectly underscored core message integrity mechanisms.

Week 6: Hashing and Password Security

Implementation of Secure Hash Functions and Authentication

Fig 1: SHA-256 Hash Generation

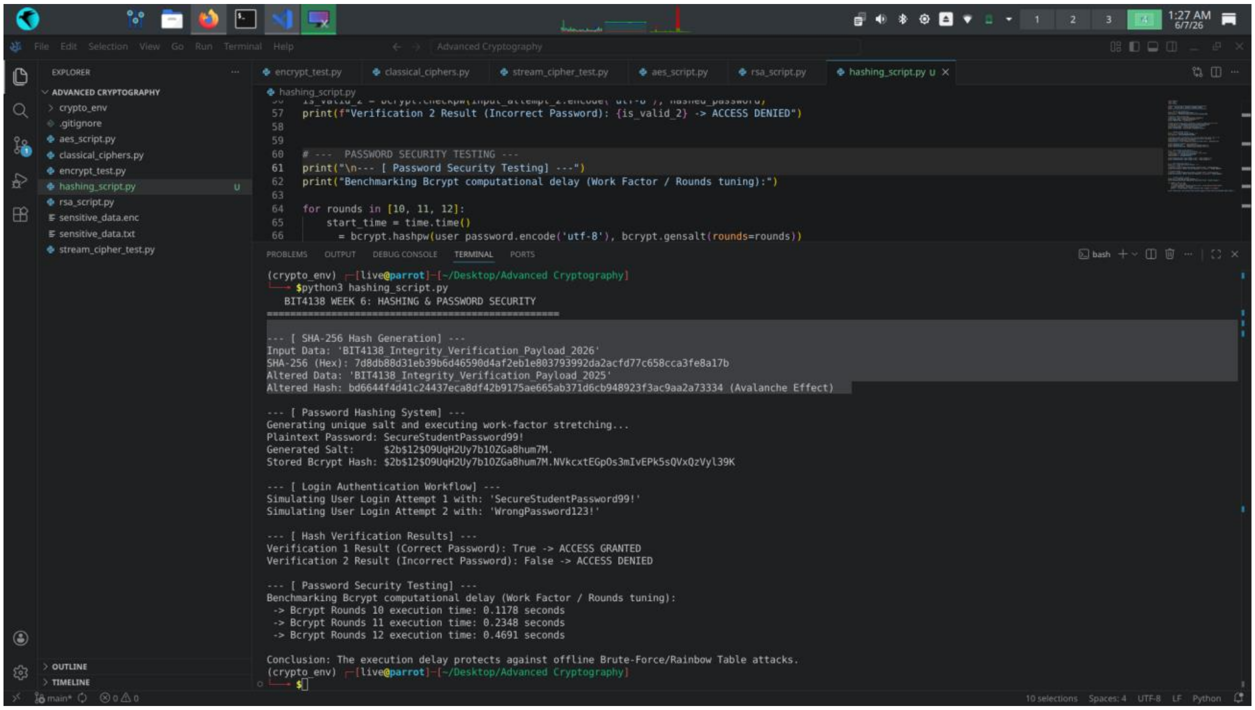


Fig 2: Password Hashing System

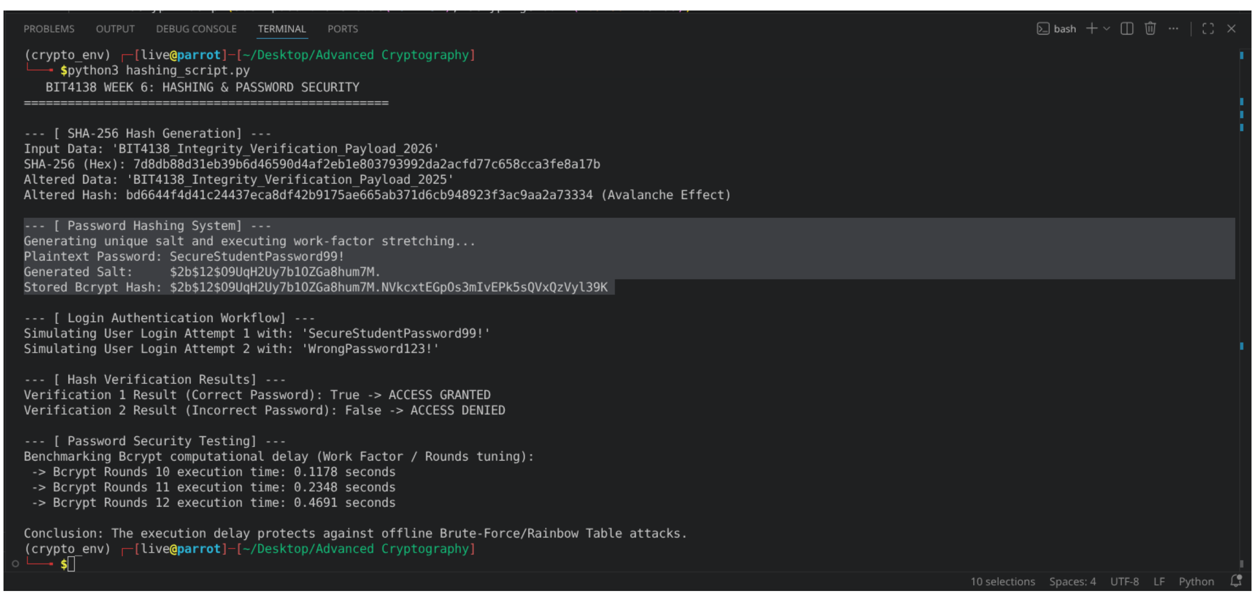
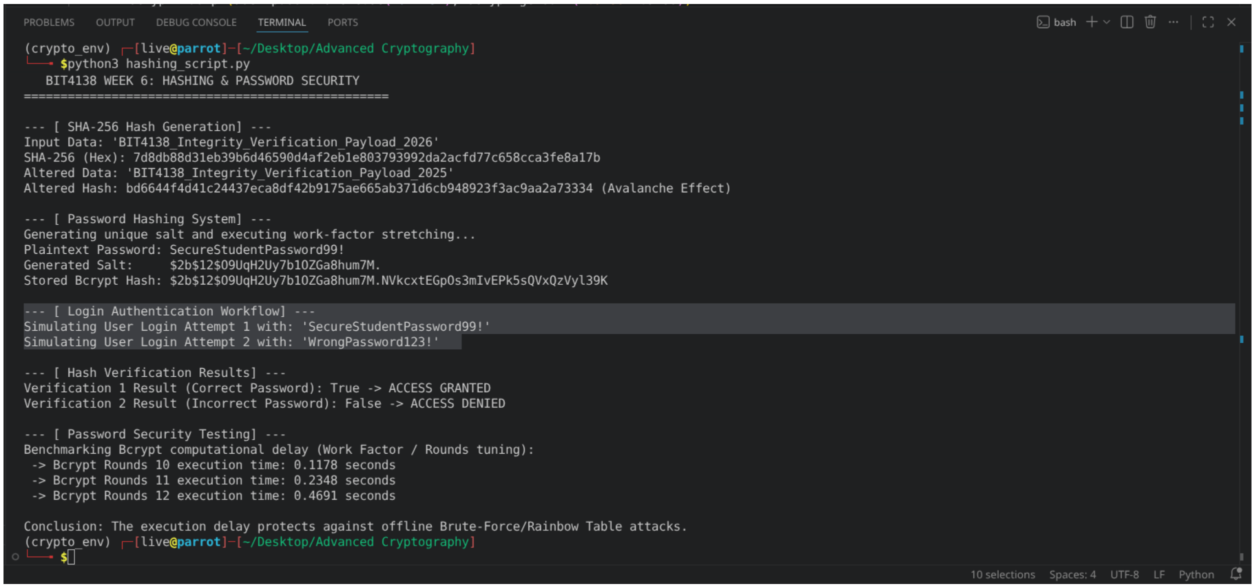
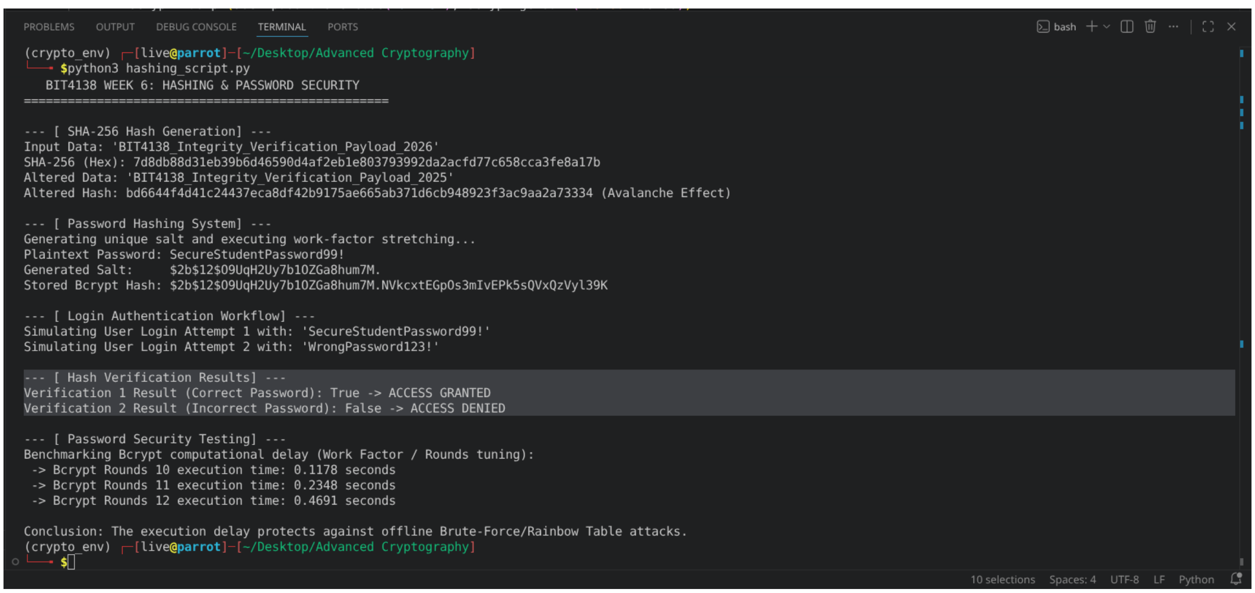


Fig 3: Login Authentication Workflow



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash
(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $python3 hashing_script.py
   BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
==================================================

--- [ SHA-256 Hash Generation] ---
Input Data: 'BIT4138_Integrity_Verification_Payload_2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138_Integrity_Verification_Payload_2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt:     $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyl39K

--- [ Login Authentication Workflow] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
 -> Bcrypt Rounds 10 execution time: 0.1178 seconds
 -> Bcrypt Rounds 11 execution time: 0.2348 seconds
 -> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $
                                          10 selections   Spaces: 4   UTF-8   LF   Python
```

Fig 4: Hash Verification Results



```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   bash
(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $python3 hashing_script.py
   BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
==================================================

--- [ SHA-256 Hash Generation] ---
Input Data: 'BIT4138_Integrity_Verification_Payload_2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138_Integrity_Verification_Payload_2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt:     $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyl39K

--- [ Login Authentication Workflow] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
 -> Bcrypt Rounds 10 execution time: 0.1178 seconds
 -> Bcrypt Rounds 11 execution time: 0.2348 seconds
 -> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $
                                          10 selections   Spaces: 4   UTF-8   LF   Python
```

Fig 5: Password Security Testing

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                        bash

(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $python3 hashing_script.py
   BIT4138 WEEK 6: HASHING & PASSWORD SECURITY
==================================================

--- [ SHA-256 Hash Generation] ---
Input Data: 'BIT4138_Integrity_Verification_Payload_2026'
SHA-256 (Hex): 7d8db88d31eb39b6d46590d4af2eb1e803793992da2acfd77c658cca3fe8a17b
Altered Data: 'BIT4138_Integrity_Verification_Payload_2025'
Altered Hash: bd6644f4d41c24437eca8df42b9175ae665ab371d6cb948923f3ac9aa2a73334 (Avalanche Effect)

--- [ Password Hashing System] ---
Generating unique salt and executing work-factor stretching...
Plaintext Password: SecureStudentPassword99!
Generated Salt:     $2b$12$09UqH2Uy7b10ZGa8hum7M.
Stored Bcrypt Hash: $2b$12$09UqH2Uy7b10ZGa8hum7M.NVkcxtEGp0s3mIvEPk5sQVxQzVyl39K

--- [ Login Authentication Workflow] ---
Simulating User Login Attempt 1 with: 'SecureStudentPassword99!'
Simulating User Login Attempt 2 with: 'WrongPassword123!'

--- [ Hash Verification Results] ---
Verification 1 Result (Correct Password): True -> ACCESS GRANTED
Verification 2 Result (Incorrect Password): False -> ACCESS DENIED

--- [ Password Security Testing] ---
Benchmarking Bcrypt computational delay (Work Factor / Rounds tuning):
 -> Bcrypt Rounds 10 execution time: 0.1178 seconds
 -> Bcrypt Rounds 11 execution time: 0.2348 seconds
 -> Bcrypt Rounds 12 execution time: 0.4691 seconds

Conclusion: The execution delay protects against offline Brute-Force/Rainbow Table attacks.
(crypto_env) ┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $

                                        10 selections   Spaces: 4   UTF-8   LF   Python
```

Creative Task: Password Strength Analyzer and Leak Simulator

Fig 1: Password Entropy and Failure Mode

```
==================================================
🕵️LAB ANALYSIS FOR INPUT: 'P@ss_W0rd_Extr3m3_2026'
==================================================

[STEP 1: ENTROPY & STRUCTURE CHECK]
 - Length >= 10 chars: ✅ PASSED
 - Contains Numbers: ✅ PASSED
 - Contains Uppercase: ✅ PASSED
 - Contains Special Char: ✅ PASSED
👉 Structural Strength Assessment: STRONG (4/4 criteria met)
```

Fig 2: Simulated Breach / Rainbow Table Match

```
[STEP 2: CRYPTOGRAPHIC HASH GENERATION]
 - Computed SHA-256 Hash: b01b419134f1fc82ad80b348645f545e43f647d546a9c81f29130da0b3566a36
```

Fig 3: Secure Cryptographic Isolation

```
[STEP 3: SIMULATED BREACH & RAINBOW TABLE LOOKUP]
🛡️No matches found in the leak database. Password is structurally isolated.
```

Student Reflection

Evaluated SHA-256 data integrity tracking and implemented a resilient password storage system using bcrypt. I explored how salting neutralizes precomputed rainbow table vector threats while algorithmic stretching stops brute-force tools. Mitigating type errors during byte-string formatting proved challenging, but the metrics validated how computational latency properly defends user credentials.

Week 7: Digital Signatures and Certificates

Digital Signature Implementation and Certificate Management

Fig 1: Digital Signature Generation

```
┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $python3 signature_system.py
---  SIGNATURE GENERATION ---
Document Message: Academic Logbook Data: Integrity Verified.
Generated PSS Signature (Hex): c640736af9c82cba8618ce16f96ed4371e5de37b746744c889ec9a7c870a7b26...
```

Fig 2: Signature Verification Process

```
# 4. Signature Verification Process
print("\n--- SIGNATURE VERIFICATION & VALIDATION ---")
try:
    public_key.verify(
        signature,
        document_data,
        padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=padding.PSS.MAX_LENGTH),
        hashes.SHA256()
    )
    print("[SUCCESS] Signature matches document. Integrity & Authenticity confirmed!")
except Exception:
    print("[FAILURE] Signature validation failed.")
```

Fig 3: Certificate Creation Using OpenSSL

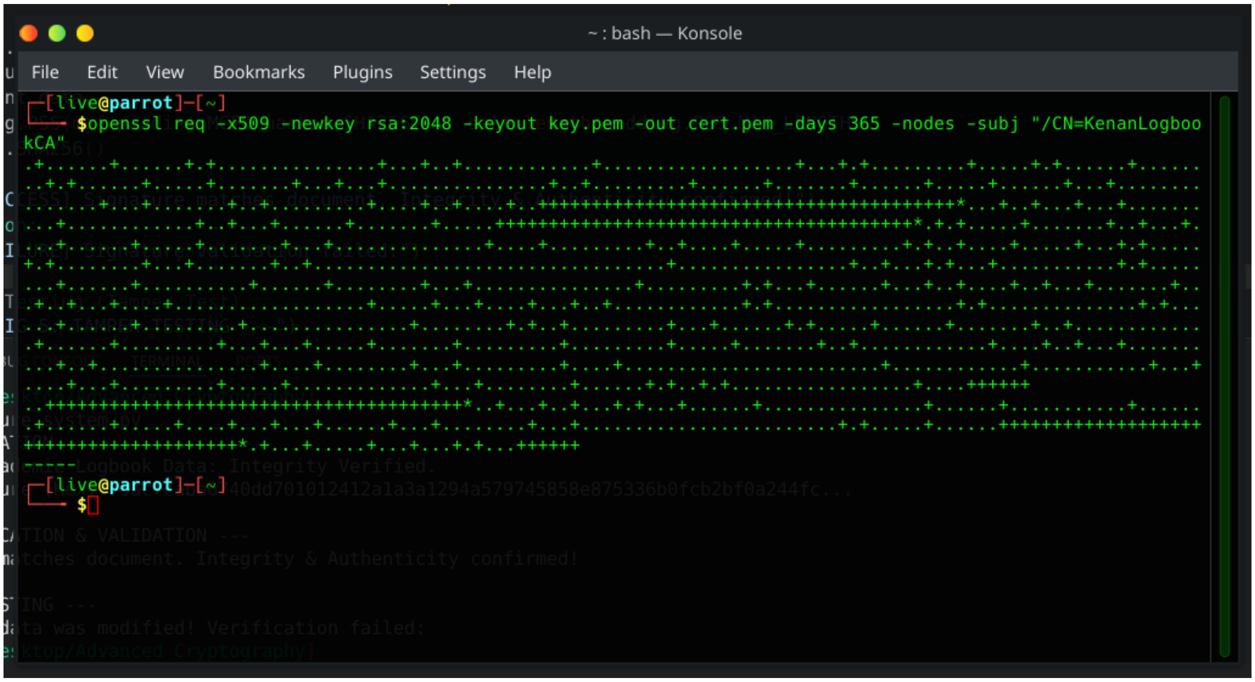


Fig 4: Secure Document Validation

```
--- SIGNATURE VERIFICATION & VALIDATION ---
[SUCCESS] Signature matches document. Integrity & Authenticity confirmed!
```

Fig 5: Certificate Testing Results

```
--- TAMPER TESTING ---
[REJECTED] Document data was modified! Verification failed:
┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $
```

Creative Task: Software Update Integrity Guard

Fig 1: Cryptographic Packaging and Core Signing Execution

```
┌─[live@parrot]─[~/Desktop/Advanced Cryptography]
└──╼ $python3 software_guard.py

[STAGE 1: GENERATING DEVELOPER CRYPTOGRAPHIC IDENTITIES]
✅ Keypair generated successfully (RSA 2048-bit).

[STAGE 2: PACKAGING AND SIGNING SOFTWARE UPDATE]
 - Software Update File Contents: b"print('System updating... Installing Security Patches v4.1.2')"
 - Generated Vendor Digital Signature: 6c89c5cd1e74244cfadb967e4212bfb4e6d9f03b5a02ce220f...
📦 Update Package ready for deployment.
```

Fig 2: Client Verification Success & Deployment Authorization

```
[STAGE 3: CLIENT SIMULATION - VALIDATING AUTHENTIC UPDATE]
✅ VERIFICATION SUCCESSFUL: Signature matches developer certificate.
🚀 ACTION: Executing verified code payload safely...
```

Fig 3: Malicious Interception and Tamper Defiance

```
[STAGE 4: THREAT EMULATION - INJECTING MALICIOUS PAYLOAD]
⚠️Attacker swapped payload to: b"print('System hacked! Executing malware payload...')"
🔍 Client Endpoint scanning update metadata...
🚨 SECURITY ALERT: DIGITAL SIGNATURE MISMATCH OR CERTIFICATE UNTRUSTED!
🛑 ACTION: Operation aborted. Dropping corrupt software package to protect kernel.
```

Student Reflection

Digital signatures guarantee non-repudiation and data integrity. The authentication workflow binds a public key to an identity via certificates. Validation checks validity periods and cryptographic signatures against trusted roots. Real-world applications include secure e-commerce, software distribution signing, code signing, and securing HTTPS communications using SSL/TLS infrastructure globally.

Week 8: Public Key Infrastructure

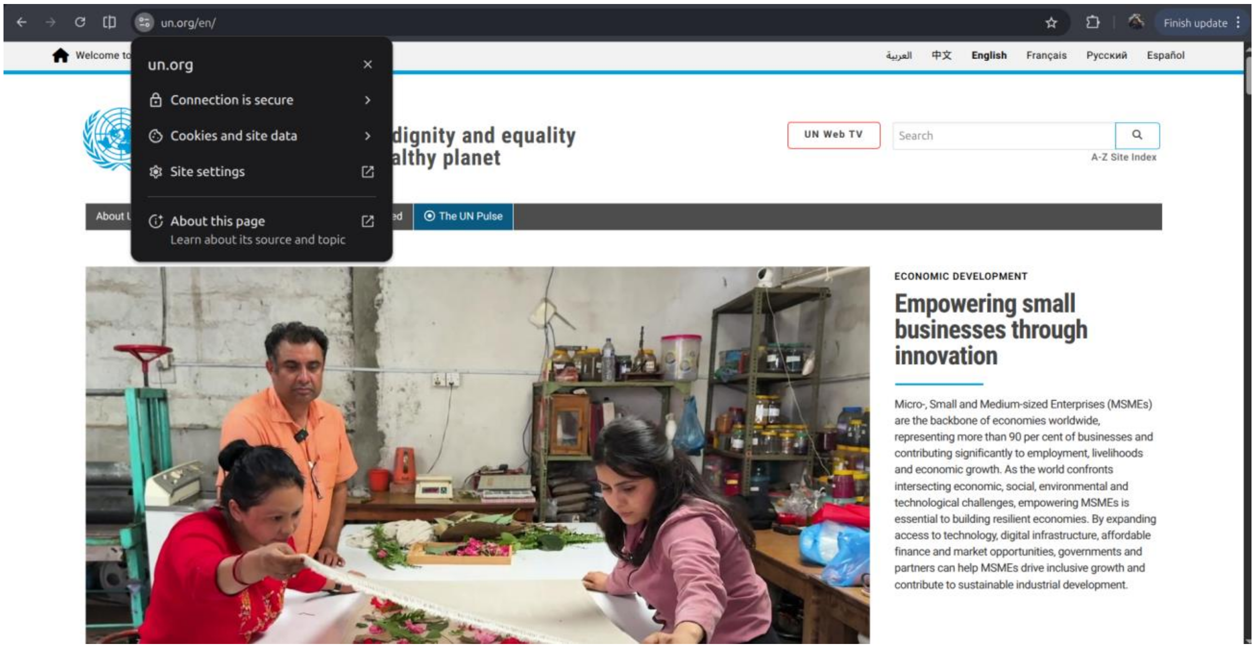
Part 1: PKI and Digital Certificates

Class Activity 1: PKI Component Identification

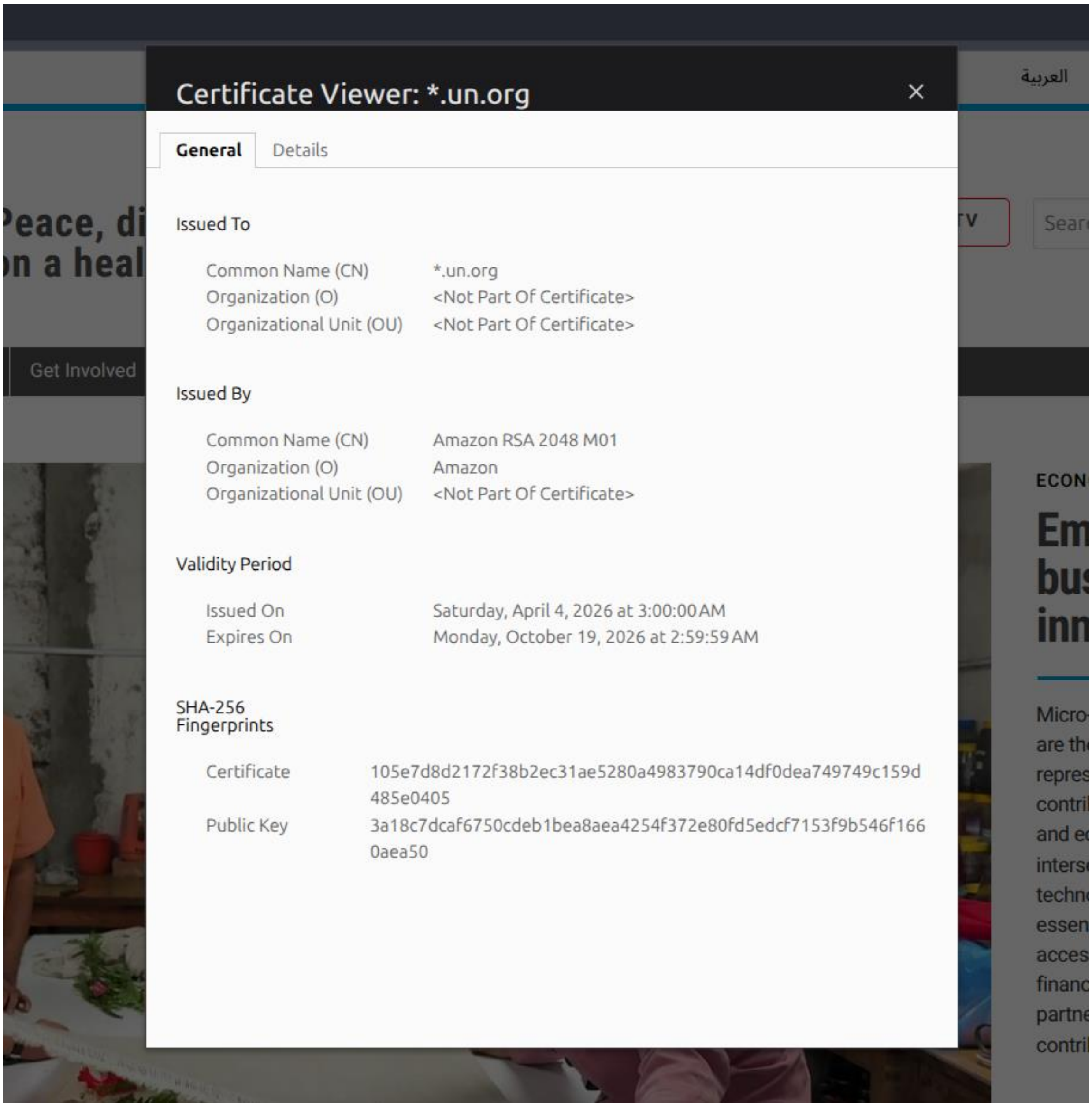
a) Certificate Authority (CA): A trusted organization that verifies identities, issues digital certificates, and manages their revocation.
b) Registration Authority (RA): An intermediary that verifies user identity and approves certificate requests before sending them to the CA.
c) Digital Certificate: An electronic document binding an entity's identity to its public key.
d) End User: Individuals, organizations, or systems (like web browsers) that use certificates to establish trust.
e) Certificate Repository: A database where issued certificates are stored and accessed.

Class Activity 2: Certificate Analysis

Selected Website: United Nations Website

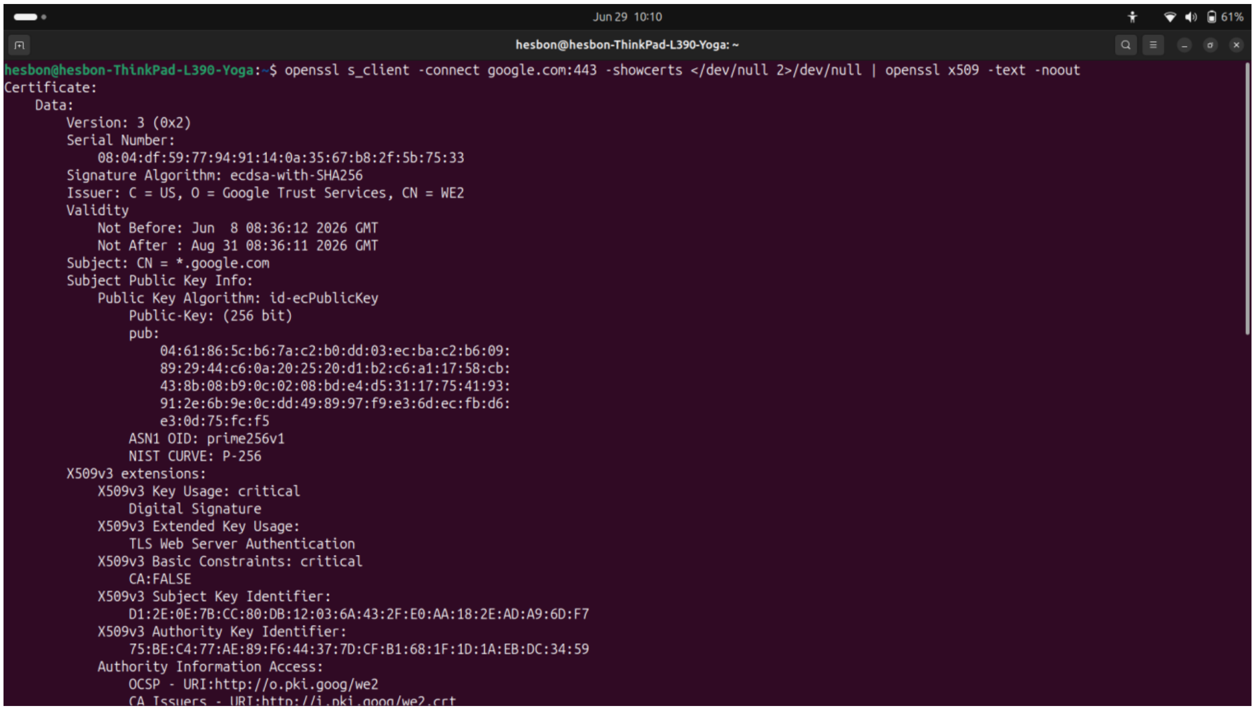


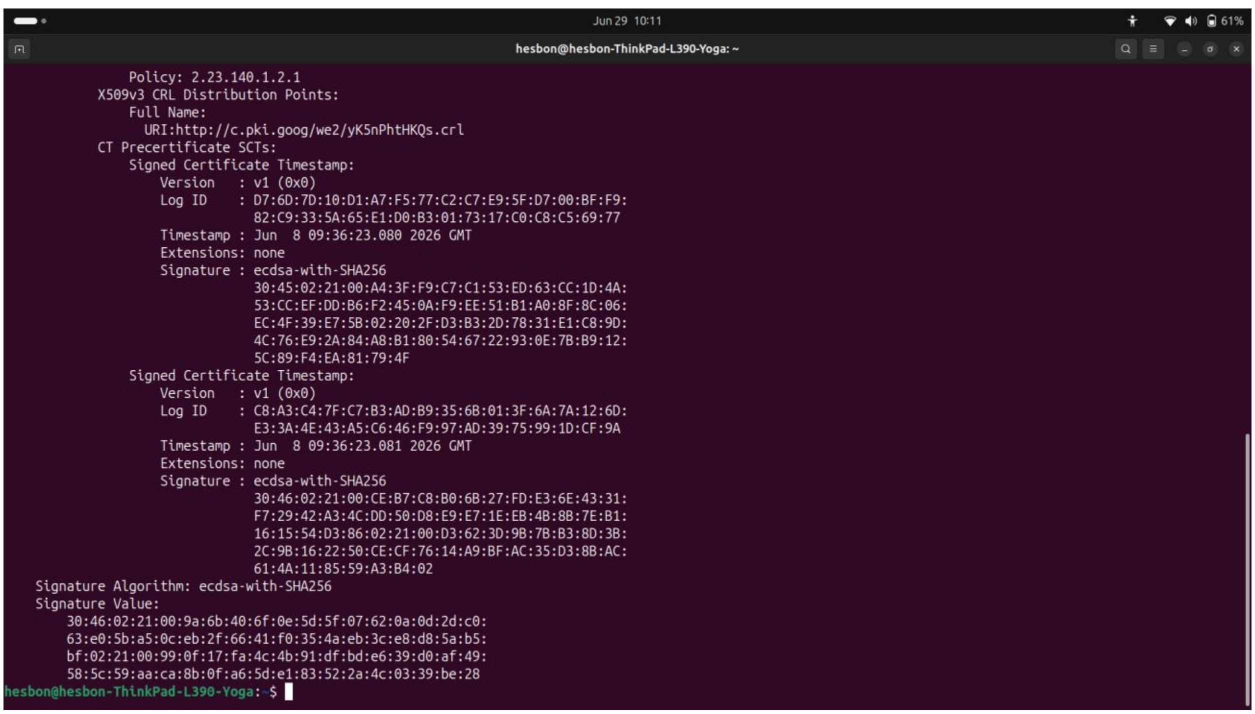
Certificate Information



OpenSSL Certificate Inspection

Google.com Certificate Inspection





Digital Certificate Security Incident Report

The DigiNotar breach occurred when Iranian hackers compromised the CA, issuing over 500 fraudulent certificates, including one for Google. This allowed man-in-the-middle attacks spying on thousands of users. Consequently, browsers untrusted DigiNotar, forcing the company into bankruptcy. This underscored the need for certificate pinning and automated revocation methods.

Part 2: Asymmetric Ciphers and Diffie-Hellman

Class Exercise 1: Understanding Asymmetric Cryptography

a) Public Key: A publicly distributed key used to encrypt data or verify digital signatures.

b) Private Key: A strictly confidential key kept by the owner to decrypt data or generate digital signatures.

c) Symmetric Encryption: A system using a single shared key for both encryption and decryption.

d) Asymmetric Encryption: A system using a mathematically linked public-private key pair.

Class Exercise 2: Complexity and Cryptography

Computationally hard problems ensure that while encryption and decryption are fast for authorized parties, reversing the process without the key is computationally infeasible for attackers. Three examples include:

1. Discrete Logarithm Problem (used in Diffie-Hellman).
2. Integer Factorization (used in RSA).
3. Elliptic Curve Discrete Logarithm Problem (used in ECC).

Practical Task: Implementing Diffie-Hellman Key Exchange

Assignment 2: Diffie-Hellman in Modern Internet Security Report

The Diffie-Hellman protocol establishes shared secrets over insecure channels using the mathematically intractable Discrete Logarithm Problem. Essential to TLS/SSL handshakes, it secures HTTPS and VPN connections. Because it lacks built-in authentication, it is paired with digital signatures to prevent devastating Man-in-the-Middle exploits.

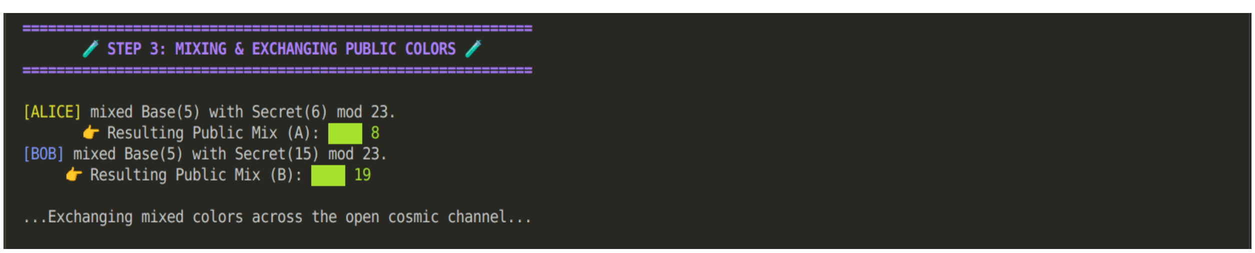
Creativity Assignment

A python script that models the **Diffie-Hellman Paint Analogy** visually in your terminal using colored text blocks.

Step 1:

Step 2:

Step 3:



Step 4:

```
============================================================
    🎨 STEP 4: COMPUTING THE FINAL SHARED SECRET SHADE 🎨
============================================================

[ALICE] injects her SecretFactor(6) into Bob's Mix(19).
       Final Shared Secret Paint Code: ████ 2
[BOB] injects his SecretFactor(15) into Alice's Mix(8).
     Final Shared Secret Paint Code: ████ 2

🎉 SUCCESS: Secure Cryptographic Key Connection Verified!
```

Student Reflection

This practical assignment taught me how Public Key Infrastructure (PKI) manages digital certificates to secure communications. I also implemented a Python script for the Diffie-Hellman protocol, discovering how the intractable Discrete Logarithm Problem allows secure key exchanges over open channels without revealing private keys.

Week 9: RSA Cryptosystem

Part A: Key Generation

```
○ hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week-9$ python3 main_menu_app.py

=================================
      RSA SECURITY SYSTEM
=================================
1. Generate RSA Keys
2. Encrypt Message
3. Decrypt Message
4. Sign Message
5. Verify Signature
6. Miller-Rabin Prime Test
7. Exit
=================================
Enter selection (1-7): 1

--- Part A: Key Generation ---
Select operation mode ([S]ecure 512-bit / [W]eak 16-bit for lab cracking demonstration): S

[+] Prime P:       12608104057731176476835686738316210389606130406231541907839326680736703913363000992942955774034793150154573350054334088180144080084152708016417938634320717
[+] Prime Q:       7742755538372382683073153429046372023489134310380201803698565749512879462945921527144397396326616820986975683316214381604588169301177164434417560549591917
[+] Modulus N:     976214675213733779987582906439180679840609424502321699555113761173819961834714840699761400598052874735885631889066407936075607041796588230058302183705432848636049
14593510819168996773787957681846149030929073454126280081520070173350371685896103961903045515797360716682588960439943648573742328503815204848844489
[+] Euler Totient: 976214675213733779987582906439180679840609424502321699555113761173819961834714840699761400598052874735885631889066407936075607041796588230058302183705432645127453
18489951659260156606425375268750884314317329742588387651270486797041449165808750791541635544655811683312040490655211399188412456052979705664931856
[+] Public Key e:  65537
[+] Private Key d: 133554187374556923133019032594316095876388972642748620751806612794062434621817496402246985943240338661854381120853402101940184822874837268576571057251673239486286
30019392199473985140198817685576398504084245212201466098254286656732435619888631759112597529538794994469929277190207446253617133542441918931165281
```

Part B: Encryption

```
==================================
       RSA SECURITY SYSTEM
==================================
1. Generate RSA Keys
2. Encrypt Message
3. Decrypt Message
4. Sign Message
5. Verify Signature
6. Miller-Rabin Prime Test
7. Exit
==================================
Enter selection (1-7): 2

--- Part B: Character Encryption ---
Enter plaintext string (Default: HELLO WORLD): Commas Ayra Starr

Original    ASCII     Encrypted Integer Value
-------------------------------------------------------------
C           67        108323224755409405245467970829...
o           111       127081273701458635696687181779...
m           109       104688942578962688248315806657...
m           109       104688942578962688248315806657...
a           97        735274309822297650213823943199...
s           115       111260119530791786121708707543...
            32        121658197371617539862346692358...
A           65        126632949664152914206059699089...
y           121       426072479409319848476402942961...
r           114       100277326029706005316317409107...
a           97        735274309822297650213823943199...
            32        121658197371617539862346692358...
S           83        822720528112275960226672684282...
t           116       128518985686706346892672853357...
a           97        735274309822297650213823943199...
r           114       100277326029706005316317409107...
r           114       100277326029706005316317409107...
```

Part C: Decryption

```
--- Part C: Character Decryption ---

Encrypted Block (Truncated) -> Decrypted ASCII    -> Character
------------------------------------------------------------------
108323224755409405 24...   -> 67                 -> C
127081273701458635 69...   -> 111                -> o
104688942578962688 24...   -> 109                -> m
104688942578962688 24...   -> 109                -> m
735274309822297650 21...   -> 97                 -> a
111260119530791786 12...   -> 115                -> s
121658197371617539 86...   -> 32                 ->
126632949664152914 20...   -> 65                 -> A
426072479409319848 47...   -> 121                -> y
100277326029706005 31...   -> 114                -> r
735274309822297650 21...   -> 97                 -> a
121658197371617539 86...   -> 32                 ->
822720528112275960 22...   -> 83                 -> S
128518985686706346 89...   -> 116                -> t
735274309822297650 21...   -> 97                 -> a
100277326029706005 31...   -> 114                -> r
100277326029706005 31...   -> 114                -> r

[+] Recovered Clean Output: Commas Ayra Starr
```

Part D: Digital Signature

```
--- Part D: Digital Signature Generation ---
Enter transaction payload or message to sign: Sign This Message
[+] Computed SHA-256 Digest: 8c4e3ef5043e6b1e9603b4a67b946a1094db75295645978b1dc151472c095e13
[+] Output Signature Value:  60921474796285884267746603458519066046466845455321696156435353113248612718952820226281133496768711805589647343720494216324773386106572965285116073560704
82833879609265601078600038348449918283099331406179300651363824716364429851398445570141390178219557046296751622432991868628414353229461538509527562753586775 9
```

Part E: Signature Verification

```
--- Part E: Signature Verification ---
Simulate unauthorized payload tampering? (y/n): y
[*] Evaluated Payload String:  Sign This MessageAX
[*] Transmitted Hash Hex:       58f6291da30c96caa262b6768c6961f52b68b1f60efdd9eb578fa6f23573a6ce
[*] Decrypted Signature Hash:   8c4e3ef5043e6b1e9603b4a67b946a1094db75295645978b1dc151472c095e13

==============================
```

Part F: Miller-Rabin Test

```
--- Part F: Miller-Rabin Prime Verification Loop ---
Generated Number  Classification Status
----------------------------------------
43                Probably Prime
64                Composite
158               Composite
1489              Probably Prime
979               Composite
3604              Composite
4336              Composite
4290              Composite
495               Composite
2634              Composite
```

Creative Challenge

Sending Hello To Bob From Alice

```
○ hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week-9$ python3 rsa_secure_chat.py
========================================
      WELCOME TO RSA SECURE CHAT
========================================
[*] Generating cryptographically secure keypairs for Alice...
[*] Generating cryptographically secure keypairs for Bob...

--- CHAT NETWORKING OPERATIONS PANEL ---
1. Transmit Encrypted Message
2. Print Registered Users & Public Keys
3. Register New Participant Profile
4. Inspect Disk Storage Log Archive
5. Purge History File Logs
6. Exit Chat Console
Enter selection index (1-6): 1
Specify SENDER Identity ['Alice', 'Bob']: Alice
Specify RECIPIENT Identity ['Alice', 'Bob']: Bob
Enter plain payload text string for Bob: Hello

[+] NETWORK ENVELOPE TRANSMITTED SUCCESSFULLY:
    Cipher blocks sent: [419072912790040841382946839525071400084511362484910874779129560396601219939377131404294802338206168087470807995302694974432860714478088745439807706988819186
16414988309061773602595256342604477601437403393399390595990632652229114447892306836471676939086507618360439518018270692306446389433128144747693749936609, 600770489016442257727945133
576766965022772088693298337222862301187859532161460217592311722868822945254135839303233339728160091411546631578019883126097107317528502716072954507248739088885169653268353403659229833
1529099822989222581587161880914603789431075833540494684661200596207259528759421548293502417996768497, 228576272255186666458844074251510671243866655982776545647914848986318650119101 39
6008391915282991850726412758331099875883494664086672492022489368709501462834011054958893245066771522592560242681604224282847340900717124031073771870846675764653426973078617039118218
6769779712928095439900526256365217893262418356 04, 2285762722551866664588440742515106712438666559827765456479148489863186501191013960083919152829918507264127583310998758834946640866 72
4920224893687095014628340110549588932450667715225925602426816042242828473409007171240310737718708466757646534269730786170391182186769779712928095439900526256365217893262418356 04, 47
21292398084056743232334282969589378798932282425324573205177660354486278475625510212550107484435379112906230679704058126790636973511494883771637848359628993458844034122130092024560 43
843069041482242939625028763445283015332899101951985163377991482329700896999913321024392505076408803559334289863882843268 62806]

[+] RECIPIENT (Bob) INCOMING DECRYPTED INBOX:
    [Time Record]: 2026-06-29 12:49:09
    [Plain Payload]: Hello
    [Authentication]: Signature Verified! Authenticated from Alice.
```

Printing Registered Users & Public Keys

```
--- GLOBAL PUBLIC KEY DIRECTORY REGISTER ---
User: Alice
  Public key exponent (e): 65537
  Modulus parameter (n):   58310889977952736891052274923313858914 88...
User: Bob
  Public key exponent (e): 65537
  Modulus parameter (n):   78974132323947802876979531257540820555 61...
```

Registering new User

```
--- CHAT NETWORKING OPERATIONS PANEL ---
1. Transmit Encrypted Message
2. Print Registered Users & Public Keys
3. Register New Participant Profile
4. Inspect Disk Storage Log Archive
5. Purge History File Logs
6. Exit Chat Console
Enter selection index (1-6): 3
Input new account unique handle: Joyce
[*] Generating cryptographically secure keypairs for Joyce...
[+] Account active. Profile logged to registry cluster.
```

Reading Disk Log Archive

```
--- CHAT NETWORKING OPERATIONS PANEL ---
1. Transmit Encrypted Message
2. Print Registered Users & Public Keys
3. Register New Participant Profile
4. Inspect Disk Storage Log Archive
5. Purge History File Logs
6. Exit Chat Console
Enter selection index (1-6): 4

--- READING DISK LOG ARCHIVE (rsa_chat_history.txt) ---
Timestamp: 2026-06-29 12:49:09
Sender:    Alice
Recipient: Bob
Ciphertext Array: [4190729127900408413829468395250714000845113624849108747791295603966012199393771314042948023382061680874708079953026949744328607144780887454398077069888191861641 49
883090617736025952563426044776014374033933993905959906326522291144478923068364716769390865076183604395180182706923064463894331281447476937499366 09, 600770489016442257727945133576766
9650227720886932983372228623011878595321614602175923117228688229452541358393032333972816009141154663157801988312609710731752850271607295450724873908888516965326835340365922983152909
9822989222581587161880914603789431075833540494684661200596207259528759421548293502417996768497, 2285762722551866645884407425151067124386665598277654564791484898631865011910139600839
1915282991850726412758331099875883494664086672492022489368709501462834011054958893245066771522592560242681604224282847340900717124031073771870846675764653426973078617039118218676977
9712928095439900526256365217893262418356 04, 2285762722551866645884407425151067124386665598277654564791484898631865011910139600839191528299185072641275833109987588349466408667249 2022
4893687095014628340110549588932450667715225925602426816042242828473409007171240310737718708466757646534269730786170391182186769777971292809543990052625636521789326241835604, 47212923
980840567432323342829695893787989322824253245732051776603544862784756255102125501074844353791129062306797040581267906369735114948837716378483596289934588440341221300920245604384 3069
041482242939625028763445283015332899101951985163377991482329700896999913321024392505076408803559334289863882843268628 06]
Decrypted Payload: Hello
--------------------------------------------------
```

Purging File Logs

```
--- CHAT NETWORKING OPERATIONS PANEL ---
1. Transmit Encrypted Message
2. Print Registered Users & Public Keys
3. Register New Participant Profile
4. Inspect Disk Storage Log Archive
5. Purge History File Logs
6. Exit Chat Console
Enter selection index (1-6): 5
[!] Local history clearing execution complete.

--- CHAT NETWORKING OPERATIONS PANEL ---
1. Transmit Encrypted Message
2. Print Registered Users & Public Keys
3. Register New Participant Profile
4. Inspect Disk Storage Log Archive
5. Purge History File Logs
6. Exit Chat Console
Enter selection index (1-6): 4

--- READING DISK LOG ARCHIVE (rsa_chat_history.txt) ---
[-] Storage archive empty or log file uninitialized.
```

Student Reflection

This week, I mastered asymmetric cryptography by implementing the RSA algorithm, focusing on key generation, character encryption, and digital signatures. I also explored foundational mathematical concepts, including the Miller Rabin primality test and Pollard's Rho factorization, which taught me how modular arithmetic forms the core defense of modern data infrastructure.

Week 10: Elgamal Cryptosystem and Elliptic Curve Cryptography

Part A: Key Generation

```
===================================
     ELGAMAL SECURITY SYSTEM
===================================
1. Generate Keys
2. Encrypt Message
3. Decrypt Message
4. Compare RSA vs ElGamal (Benchmark)
5. ECC Demo Operations
6. Exit

Select an option (1-6): 1

Generating secure parameters...

[+] Success! Keys Generated:
 -> Prime Field (p): 4457
 -> Generator (g):   3
 -> Private Key (x): 3290 (Keep Secret!)
 -> Public Key (y):  1518
```

Part B: Encryption

```
Select an option (1-6): 2
Enter standard plaintext string: Plaintext String

Encrypting sequence details:
Char  ASCII   Random k  C1 Code   C2 Code
--------------------------------------------------
'P':<680      2556      2819      1505
'l':<6108     1380      1521      3449
'a':<697      429       2457      1437
'i':<6105     1103      3963      456
'n':<6110     3981      674       3136
't':<6116     1187      1563      3633
'e':<6101     2356      2362      225
'x':<6120     564       4059      560
't':<6116     2016      733       3056
' ':<632      2388      3160      4278
'S':<683      4278      292       4324
't':<6116     1658      2254      1488
'r':<6114     1554      826       1618
'i':<6105     1179      2028      40
'n':<6110     144       436       3339
'g':<6103     4218      4031      942

[+] Complete Ciphertext Payload Stored.
```

Part C: Decryption

```
Select an option (1-6): 3

Decrypting payload segments...

[+] Recovered Decrypted Message: Plaintext String
```

Part D: RSA vs ElGamal

```
Select an option (1-6): 4

Executing Cryptographic Performance Benchmark...

+----------------------------------------------------------+
| Algorithm | Key Gen (ms)  | Encrypt (ms)  | Decrypt (ms)  |
+----------------------------------------------------------+
| RSA       | 0.2067        | 0.0197        | 0.0171        |
| ElGamal   | 0.1744        | 0.0279        | 0.0321        |
+----------------------------------------------------------+

Architectural Observation Notes:
 -> ElGamal encryption requires computing two dynamic exponentiations due to random parameter k.
 -> RSA encryption with a small public exponent (e.g., 65537) processes drastically faster than asymmetric alternatives.
```

Part E: ECC Demonstration

```
Select an option (1-6): 5

Initializing Elliptic Curve Operations Demo...
Using Curve Configuration: y^2 = x^3 + 1x + 6 mod 11
Base Generator Point G = (2, 7)
 -> Point Doubling (2G) calculation: (5, 2)
 -> Point Addition (G + 2G = 3G):    (8, 3)
 -> Scalar Multiplication (4 * G) [Public Key Point]: (10, 2)
[Hardness Context] Knowing G and Public point, extracting the raw multiplier scalar is computationally infeasible on cryptographically large primes.
```

Creative Challenge

Logging in

```
--- Authentication Firewall Identity Prompt ---
Enter Identification Handler Account: student01
Enter Secure Passphrase Configuration: compSci2026
[+] Authentication Verified. Logged into terminal as: Student
============================================================
          SECURE CAMPUS MESSENGER PLATFORM
         [Environment Security Node Active]
============================================================
Active Account Session Context: student01 | Role: Student
1. Write/Transmit Secure Cipher Packet
2. Display Message Vault Entries
3. Query/Search Log Text Matrices
4. Export System Workspace Archive Logs
5. Logout Active Context Session

Execute option action:
```

Transmitting Message

```
--- Write Encrypted & Digitally Signed Communications ---
Identify target receiver handler identifier: lecturer01
Type confidential message body payload text: This is a very confidential message
[+] Message successfully secured, signed via private cryptographic vector and committed.
```

Displaying Secured Message

```
--- Inbound/Outbound Secure Message Vault Registry ---

[Packet Entry Reference #0]
 Timestamp : 2026-06-29 13:27:42
 Originator: student01 (Student)
 Target    : lecturer01
 Payload   : This is a very confidential message
 SHA Hash  : 0333e54bb85269d0ca27582ff63f60fa5323a53b022ee88ba89c60c1e053bede
 Security Status: INVALID / COMPROMISED SIGNATURE
```

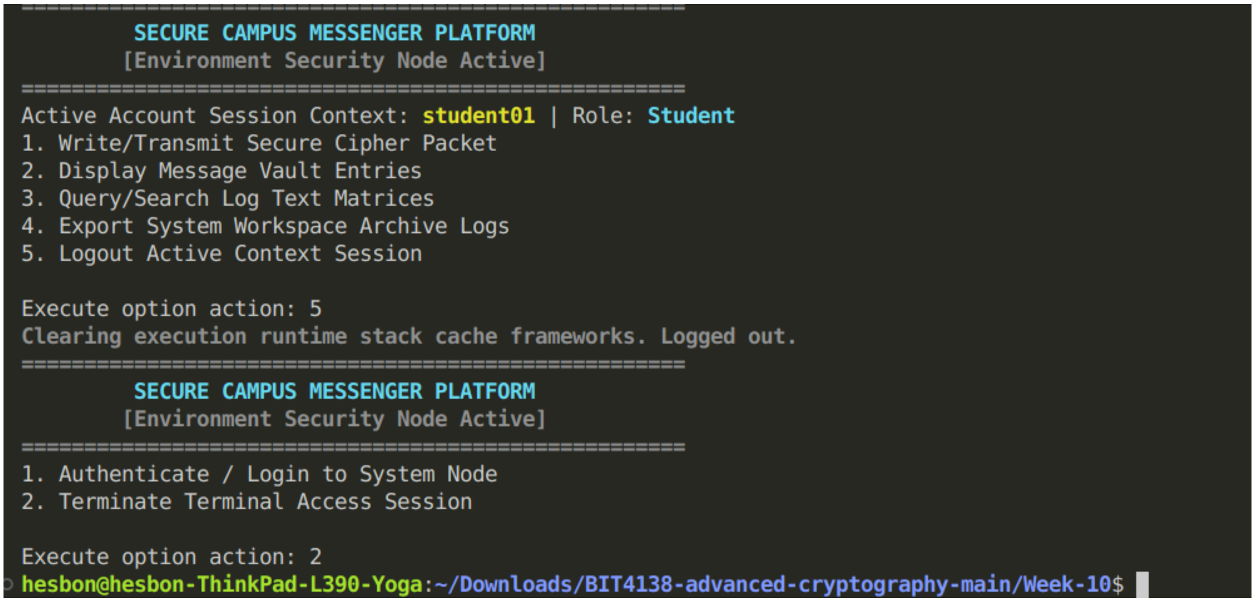
Searching for Terms

```
--- Search Message Vault Indices ---
Enter search term filter: very

Displaying matching elements for 'very':
 [2026-06-29 13:27:42] student01 -> lecturer01: This is a very confidential message
```

Exporting Logs

```
        SECURE CAMPUS MESSENGER PLATFORM
       [Environment Security Node Active]
=====================================================
Active Account Session Context: student01 | Role: Student
1. Write/Transmit Secure Cipher Packet
2. Display Message Vault Entries
3. Query/Search Log Text Matrices
4. Export System Workspace Archive Logs
5. Logout Active Context Session

Execute option action: 4

--- File System Storage Export Utility ---
[+] Local export complete. Data persisted inside file architecture: messenger_dump_student01.txt
```

Logging Out and Terminating Session



```
=======================================================
        SECURE CAMPUS MESSENGER PLATFORM
        [Environment Security Node Active]
=======================================================
Active Account Session Context: student01 | Role: Student
1. Write/Transmit Secure Cipher Packet
2. Display Message Vault Entries
3. Query/Search Log Text Matrices
4. Export System Workspace Archive Logs
5. Logout Active Context Session

Execute option action: 5
Clearing execution runtime stack cache frameworks. Logged out.
=======================================================
        SECURE CAMPUS MESSENGER PLATFORM
        [Environment Security Node Active]
=======================================================
1. Authenticate / Login to System Node
2. Terminate Terminal Access Session

Execute option action: 2
hesbon@hesbon-ThinkPad-L390-Yoga:~/Downloads/BIT4138-advanced-cryptography-main/Week-10$
```

Student Reflection

Implementing ElGamal and Elliptic Curve Cryptography this week provided valuable insights into asymmetric security frameworks. Constructing the benchmark highlighted the critical performance trade-offs between RSA's speed and ElGamal's randomized ciphertexts. Developing the secure messenger bridged abstract algebraic mathematics with practical software engineering, demonstrating how public-key infrastructure authenticates and protects campus communication.